

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-a70d7024b9da3e72a.jsonl`  
- SHA-256: `c36bdb4c76ad8859c46025d05b37301385899459f7b18a69d6cca7e5ba168f27`  
- Source modified: `2026-07-07T15:18:57+00:00`  
- Imported at: `2026-07-09T08:32:23+00:00`  
- project: `wf_a8f43370-4cb`  
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`
```

Transcript

1. user (2026-07-07T15:07:26.826Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

```
- BACKEND (repo undefined, branch undefined, PR #undefined):  
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,  
    fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote  
  the  
    choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-  
  backed  
  AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-  
  process as  
    before). Plus 4 tests in tests/test_api_cloud_dispatch.py.  
- SPA (repo undefined, branch undefined, PR #undefined):  
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when  
  capabilities.deployment.compute_target=="cloud_run" (else keeps the free "local" default).  
Plus 1 test.
```

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

```
- undefined/backend.diff (backend commit diff)  
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)  
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)  
- undefined/spa.diff (SPA commit diff)  
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)
```

You MAY read other repo files for ground truth, e.g. under undefined:

```
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;  
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;  
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);  
and under undefined: web/src/api/types.ts, web/src/api/client.ts,  
web/src/components/IngestPanel.test.tsx.
```

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

=== YOUR DIMENSION: weights-resolution-fidelity ===

Does `_local_detector_weights_present(cfg, seg_name)` FAITHFULLY mirror the REAL weights resolution the in-process runner/healthcheck uses, so it never (a) false-BLOCKS a `compute=local` run that would actually succeed, nor (b) false-ALLOWS a run that then dies with the cryptic "weights not found"? Cross-check against `native_wasb_runner.py` (env `WASB_WEIGHTS` or `indexing.wasb.weights_path`, `expanduser+exists`), `wasb_runner.py`, `tracknet_runner.py` (`indexing.tracknet.weights_path`), and `fusion_hybrid.py` (substrate `wasb|tracknet`; note there is NO native TrackNet runner). Consider: `tracknet` default `weights_path=""` ; `fusion` `substrate=tracknet` ; the WSL runner path used by `tracknet_hybrid` on a Linux control-plane ; `cfg` passed as a dict vs typed `AppConfig` ; `expanduser` vs the `/api/capabilities` probe which does NOT `expanduser` (`backend/main.py:397`) ? is that inconsistency a real bug for either the guardrail or the SPA signal?

2. assistant (2026-07-07T15:07:34.732Z)

I'll start by exploring the environment to locate the snapshot files and the real repo code, since the task has "undefined" placeholders.

3. user (2026-07-07T15:07:37.479Z)

=== cwd ===

/c/Users/avidu/Projects/khelsutra-guru/khelsutra

=== working dir listing ===

total 62

drwxr-xr-x	1	avidu	197609	0	Jul	5	12:57	.
drwxr-xr-x	1	avidu	197609	0	Jul	7	19:57	..
drwxr-xr-x	1	avidu	197609	0	Jun	22	11:12	.claude
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:33	.git
-rw-r--r--	1	avidu	197609	297	Jun	20	23:27	.gitattributes
drwxr-xr-x	1	avidu	197609	0	Jun	20	12:43	.github
-rw-r--r--	1	avidu	197609	323	Jun	22	14:57	.gitignore
drwxr-xr-x	1	avidu	197609	0	Jul	2	21:13	deploy
drwxr-xr-x	1	avidu	197609	0	Jul	2	21:13	docs
drwxr-xr-x	1	avidu	197609	0	Jun	21	23:55	locales
-rw-r--r--	1	avidu	197609	9773	Jul	2	21:13	NEXT_STEPS.md
drwxr-xr-x	1	avidu	197609	0	Jun	21	15:38	node_modules
-rw-r--r--	1	avidu	197609	1469	Jun	23	00:19	README.md
drwxr-xr-x	1	avidu	197609	0	Jun	23	00:19	site
drwxr-xr-x	1	avidu	197609	0	Jul	7	19:45	web

=== parent ===

total 108

drwxr-xr-x	1	avidu	197609	0	Jul	7	19:57	.
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:24	..
drwxr-xr-x	1	avidu	197609	0	Jun	23	09:53	Annotation Setup
-rw-r--r--	1	avidu	197609	905	Jul	2	21:20	AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:14	badminton-highlight-indexer
-rw-r--r--	1	avidu	197609	1222	Jul	2	21:35	bash.exe.stackdump
drwxr-xr-x	1	avidu	197609	0	Jul	7	20:34	bhi-wt-worker-reval
drwxr-xr-x	1	avidu	197609	0	Jul	5	12:57	khelsutra
drwxr-xr-x	1	avidu	197609	0	Jul	5	18:47	khelsutra-screencast
drwxr-xr-x	1	avidu	197609	0	Jul	5	13:44	marketing
drwxr-xr-x	1	avidu	197609	0	Jul	5	13:43	pr-158-screenshots
drwxr-xr-x	1	avidu	197609	0	Jun	22	15:03	rally-annotator
drwxr-xr-x	1	avidu	197609	0	Jul	4	06:58	sports-data-collector
drwxr-xr-x	1	avidu	197609	0	Jul	2	19:46	sports-obsreport

4. user (2026-07-07T15:07:53.577Z)

=== search for snapshot files ===

C:/Users/avidu/AppData/Local/Temp/backend.diff
C:/Users/avidu/AppData/Local/Temp/spa.diff

5. assistant (2026-07-07T15:08:00.114Z)

Found the diffs in Temp. Let me locate all snapshot files and read the key ones.

6. user (2026-07-07T15:08:03.069Z)

```
1      commit 4565c7436ccf0b4e30f68f01efe6082eff371a5d
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:56:22 2026 +0530
4
5          fix(serving): fail fast when compute=local forces weightless in-process detection
6
7          A hosted cloud-serving control-plane bakes NO detector weights into its image
8          (only the worker Cloud Run Job GCSFuse-mounts them at /gcs/models/...). So a
9          POST /api/process with compute="local" for a weights-backed trajectory detector
10         (wasb_hybrid / tracknet_hybrid / fusion_hybrid) returned None from
11         _maybe_dispatch_remote and ran the in-process segmenter ON THE CONTROL-PLANE,
12         which then died deep in the healthcheck with the confusing
13         "WASB weights not found at /opt/models/WASB-SBDT/..." (observed live 2026-07-07,
14         job f29d41d0?: [COMPUTE] requested=local actual=in_process ... status=failed).
15
16         Guardrail: on a cloud-serving-capable server (_cloud_available) whose local
17         weights for the requested detector aren't readable, a forced compute="local"
18         now fails FAST with an actionable message ("this hosted server can't run
19         detection locally ... Choose 'Run in cloud'.") ? mirroring the existing
20         compute="cloud"-not-available reject ? instead of the silent in-process trap.
21         A truly-local box, or a cloud-wired box that actually has the weights on disk,
22         is unaffected (the weights-present check is the escape hatch); a weight-free
23         segmenter (motion) is out of scope. Honest provenance: path=not_run,
24         requested=local, dispatched=False.
25
26         Covered by tests/test_api_cloud_dispatch.py (fail-fast, weights-present escape
27         hatch, motion unaffected, pure-local not guardrailed).
28
29         Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
30
31         diff --git a/backend/orchestration.py b/backend/orchestration.py
32         index 5e9f510..bfe39f5 100644
33         --- a/backend/orchestration.py
34         +++ b/backend/orchestration.py
35         @@ -155,6 +155,38 @@ def _ingest_remote_segments(
36             return n
37
38
39         ## Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS
box. A hosted
40         ## cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
41         ## GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
42         ## guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
43         ## "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
44         _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
45         +
46         +
47         +def _local_detector_weights_present(cfg, seg_name: str) -> bool:
```

```

48 +     """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
49 +     on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
50 +     and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
51 +     otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
52 +     A non-weights segmenter returns True (nothing local to check)."""
53 +     name = (seg_name or "").strip().lower()
54 +     if name not in _WEIGHTS_BACKED_SEGMENTERS:
55 +         return True
56 +     idx = getattr(cfg, "indexing", None)
57 +     substrate = (
58 +         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
59 +     ).strip().lower()
60 +     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
61 +         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
62 +     else: # wasb_hybrid, or fusion on the (default) WASB substrate
63 +         path = (
64 +             os.environ.get("WASB_WEIGHTS")
65 +             or getattr(getattr(idx, "wasb", None), "weights_path", "")
66 +             or ""
67 +         )
68 +     return bool(path) and os.path.exists(os.path.expanduser(path))
69 +
70 +
71 +     def _cloud_available(cfg) -> bool:
72 +         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
73
74 +     @@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
75 +         )
76 +         choice = ""
77 +         if choice == "local":
78 +             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
79 +             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
80 +             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
81 +             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail
FAST with an
82 +             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
83 +             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it
runs in-process
84 +             # below exactly as before (the weights-present check is the escape hatch).
85 +             if (
86 +                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
87 +                 and _cloud_available(cfg)
88 +                 and not _local_detector_weights_present(cfg, seg_name)
89 +             ):
90 +                 return _failed(
91 +                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
92 +                     "live only on the cloud worker, not on the control-plane. Choose 'Run
in cloud'.",
93 +                     False,
94 +                 )
95 +                 return None # explicit "run on my machine" ? in-process regardless of the
server's target
96 +             if choice == "cloud":
97 +                 if not _cloud_available(cfg):

```

```

98     diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
99     index 8382234..7993b7f 100644
100     --- a/tests/test_api_cloud_dispatch.py
101     +++ b/tests/test_api_cloud_dispatch.py
102     @@ -721,6 +721,114 @@ def
test_cloud_override_survives_api_boundary_on_local_server(tmp_path, monkeypa
103         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
104
105
106     + # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
-----
107     + # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
Run Job mounts
108     + # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
there. It must fail
109     + # FAST with an actionable "run in the cloud" message ? not fall through to the in-
process segmenter
110     + # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
2026-07-07).
111     +
112     +
113     + def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
114     +     """compute='local' + a weights-backed detector on a cloud_run server with NO local
weights ?
115     +     a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
found')."""
116     +     _db, _bucket, _mp = cloud_env
117     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
118     +     m.global_config.indexing.wasb.weights_path =
"/nonexistent/wasb_badminton_best.pth.tar"
119     +     fake = _FakeCloudClient()
120     +     _inject_fake(monkeypatch, fake)
121     +
122     +     out = m._execute_processing(
123     +         m.ProcessRequest(
124     +             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
125     +         )
126     +     )
127     +     assert out["status"] == "failed"
128     +     assert "can't run detection locally" in out["message"]
129     +     assert "Run in cloud" in out["message"]
130     +     assert "weights not found" not in out["message"] # NOT the cryptic deep failure
131     +     assert fake.calls == [] # never dispatched
132     +     # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
133     +     assert out["compute"]["requested"] == "local"
134     +     assert out["compute"]["dispatched"] is False
135     +     assert out["compute"]["path"] == "not_run"
136     +
137     +
138     + def test_local_on_cloud_server_with_weights_present_runs_in_process(
139     +     cloud_env, tmp_path, monkeypatch
140     + ):
141     +     """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
ACTUALLY has the
142     +     detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
143     +     _db, _bucket, _mp = cloud_env
144     +     weights = tmp_path / "wasb_badminton_best.pth.tar"
145     +     weights.write_bytes(b"WEIGHTS")
146     +     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
147     +     m.global_config.indexing.wasb.weights_path = str(weights)
148     +     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
149     +     fake = _FakeCloudClient()
150     +     _inject_fake(monkeypatch, fake)
151     +

```

```

152 + out = m._execute_processing(
153 +     m.ProcessRequest(
154 +         video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
155 +     )
156 + )
157 + assert out["status"] == "success"
158 + assert fake.calls == [] # in-process, not dispatched
159 + assert out["compute"]["path"] == "in_process"
160 +
161 +
162 +def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
163 +    cloud_env, monkeypatch
164 +):
165 +    """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
166 +    compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
167 +    _db, _bucket, _mp = cloud_env
168 +    m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
169 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
170 +    fake = _FakeCloudClient()
171 +    _inject_fake(monkeypatch, fake)
172 +
173 +    out = m._execute_processing(
174 +        m.ProcessRequest(
175 +            video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
176 +        )
177 +    )
178 +    assert out["status"] == "success"
179 +    assert fake.calls == []
180 +    assert out["compute"]["path"] == "in_process"
181 +
182 +
183 +def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
184 +    tmp_path, monkeypatch
185 +):
186 +    """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
187 +    weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
188 +    'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
189 +    db = Database(str(tmp_path / "t.db")) # noqa: F841
190 +    monkeypatch.setattr(m, "db_instance", db)
191 +    cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa:
F841
192 +    monkeypatch.setattr(m, "global_config", cfg)
193 +    monkeypatch.setattr(
194 +        m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
195 +    )
196 +    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
197 +    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
198 +    monkeypatch.delenv("WASB_WEIGHTS", raising=False)
199 +    cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
200 +    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
201 +    video = tmp_path / "v.mp4"
202 +    video.write_bytes(b"hashable-bytes")
203 +
204 +    out = m._execute_processing(
205 +        m.ProcessRequest(
206 +            video_path=str(video), segmenter="wasb_hybrid", compute="local"
207 +        )
208 +    )
209 +    # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).

```

```

210     +   assert out["status"] == "success"
211     +   assert out["compute"]["path"] == "in_process"
212     +
213     +
214     # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
215
216
217

```

7. user (2026-07-07T15:09:16.314Z)

```

=== Temp dir snapshot files ===
-rw-r--r-- 1 avidu 197609      11027 Jul  7 20:34 backend.diff
-rw-r--r-- 1 avidu 197609      17956 Jul  7 20:34 backend_context.txt
-rw-r--r-- 1 avidu 197609      17321 Jul  7 19:52 backend_fullsuite.log
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-_ie71rq_
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-_ip1k8p8
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-1sv86gli
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-4vq6p_0g
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-6r1wobua
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-b8ps6v_t
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-eb_6zn08
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-ew0k8eif
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-fkgqcwje
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-gu0_hdvu
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-jwm3pz7m
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-o9qqpkj1
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-oajwjn11
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-qkwoiga
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-tdzu8133
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-tj7ugzqs
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-tvkf0cgr
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-uhv4turv
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-vc57digb
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-w0e0_0hf
drwxr-xr-x 1 avidu 197609         0 Jul  1 08:40 cr-ingest-yn1teez9
-rw-r--r-- 1 avidu 197609     109341 Jul  5 13:00 khelsutra-157-ingest-ux.patch
-rw-r--r-- 1 avidu 197609      53127 Jul  4 06:14 pr458.diff
-rw-r--r-- 1 avidu 197609      53127 Jul  4 06:15 pr458_full.diff
-rw-r--r-- 1 avidu 197609       5007 Jul  7 20:34 spa.diff
drwxr-xr-x 1 avidu 197609         0 Jun 22 21:46 workspaces
=== broader search ===
C:/Users/avidu/AppData/Local/Temp/backend.diff
C:/Users/avidu/AppData/Local/Temp/pr458.diff
C:/Users/avidu/AppData/Local/Temp/pr458_full.diff
C:/Users/avidu/AppData/Local/Temp/spa.diff

```

8. assistant (2026-07-07T15:09:28.157Z)

Now let me read the SPA diff, the backend context file, and locate the real repo's runner/segmenter files for ground-truth cross-checking.

9. user (2026-07-07T15:09:28.667Z)

```

1      commit 1470e2acdea4f2b478476150da61e844dce0a8e9
2      Author: Avi Dullu <avi.dullu@gmail.com>
3      Date:   Tue Jul 7 19:57:35 2026 +0530
4
5          fix(ingest): default the compute picker to CLOUD on a hosted control-plane
6
7          On a hosted cloud-serving control-plane (capabilities.deployment.compute_target
8          == "cloud_run") the engine bakes NO detector weights, so in-process LOCAL
9          detection always fails ("WASB weights not found"). The picker still defaulted to

```

```

10     LOCAL and sent compute="local", which the engine ran on the weightless
11     control-plane -> a confusing failure.
12
13     Default the picker to CLOUD when the server advertises compute_target=cloud_run
14     (cloud_available too), so the SPA never sends compute=local by default on the
15     hosted build. The free-LOCAL default is kept for a local appliance that merely
16     also offers cloud (compute_target != cloud_run). The cloud cost-acknowledgement
17     gate is unchanged (no paid dispatch without consent).
18
19     Pairs with the engine-side fast-fail guardrail (badminton-highlight-indexer
20     _maybe_dispatch_remote) for anyone who still explicitly picks LOCAL on the
21     hosted build. Covered by an IngestPanel test.
22
23     Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
24
25     diff --git a/web/src/components/IngestPanel.test.tsx
b/web/src/components/IngestPanel.test.tsx
26     index 685aa1b..b39e4f1 100644
27     --- a/web/src/components/IngestPanel.test.tsx
28     +++ b/web/src/components/IngestPanel.test.tsx
29     @@ -185,6 +185,30 @@ describe("IngestPanel", () => {
30         expect(bodies[0].compute).toBe("local");
31     });
32
33     + it("defaults to CLOUD (never sends compute=local) on a hosted cloud_run control-
plane", async () => {
34     +     // A hosted control-plane bakes no detector weights, so LOCAL can't run there ? the
picker must
35     +     // default to CLOUD. The cost gate still applies (accept it, then the sole submit
is compute=cloud).
36     +     const bodies: Record<string, unknown>[] = [];
37     +     vi.stubGlobal(
38     +         "fetch",
39     +         fetchAll({
40     +             caps: { deployment: { cloud_available: true, compute_target: "cloud_run" } },
41     +             onProcessBody: (b) => bodies.push(b),
42     +             jobs: [{ job_id: "j", status: "done", video_id: "vidH", result: { status:
"success", video_id: "vidH" } }],
43     +         }),
44     +     );
45     +     render(<IngestPanel onLoaded={vi.fn()} />);
46     +     await screen.findByTestId("compute-picker");
47     +     // Defaulted to cloud ? the cost note + ack are shown up front (LOCAL would hide
them).
48     +     expect(screen.getByTestId("compute-cost")).toBeTruthy();
49     +     typeUri("gs://bucket/match.mp4");
50     +     fireEvent.click(screen.getByRole("checkbox")); // accept the cost gate
51     +     clickProcess();
52     +     await act(async () => {});
53     +     expect(bodies).toHaveLength(1);
54     +     expect(bodies[0].compute).toBe("cloud"); // never "local" by default on the hosted
build
55     + });
56     +
57     it("shows a pasted host-path basename while processing and clears it on cancel",
async () => {
58         vi.useFakeTimers();
59         vi.stubGlobal(
60     diff --git a/web/src/components/IngestPanel.tsx b/web/src/components/IngestPanel.tsx
61     index 38ca3a9..edfd5d9 100644
62     --- a/web/src/components/IngestPanel.tsx
63     +++ b/web/src/components/IngestPanel.tsx
64     @@ -81,7 +81,9 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
65
66         // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to

```



```

a cloud GPU
67      // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
68      - // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before
it can run.
69      + // (free, no surprise cost) EXCEPT on a hosted cloud-serving control-plane
(compute_target=cloud_run):
70      + // there the control-plane bakes no detector weights, so LOCAL can never run ?
default CLOUD instead
71      + // (set from capabilities below), which requires an explicit cost acknowledgement
before it can run.
72      const [cloudAvailable, setCloudAvailable] = useState(false);
73      const [compute, setCompute] = useState<Compute>("local");
74      const [cloudAck, setCloudAck] = useState(false);
75      @@ -101,6 +103,13 @@ export function IngestPanel({ onLoaded, onBusyChange }: Props) {
76      .then((c) => {
77          if (!alive) return;
78          setCloudAvailable(c.deployment?.cloud_available === true);
79      +      // Hosted cloud-serving control-plane (compute_target=cloud_run) bakes NO
detector weights, so
80      +      // in-process LOCAL detection there always fails ("weights not found") ?
default the picker to
81      +      // CLOUD so the SPA never sends compute=local by default on the hosted build. A
local appliance
82      +      // that merely also offers cloud (compute_target?cloud_run) keeps the free
LOCAL default.
83      +      if (c.deployment?.cloud_available === true && c.deployment?.compute_target ===
"cloud_run") {
84      +          setCompute("cloud");
85      +      }
86      setBackends(c.storage?.backends ?? []);
87      setMaxUploadBytes(
88          typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb
* 1024 * 1024 : undefined,
89

```

10. user (2026-07-07T15:09:29.238Z)

```

1      return n
2
3
4      # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
5      # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
6      # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
7      # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
8      # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
9      _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
10
11
12      def _local_detector_weights_present(cfg, seg_name: str) -> bool:
13          """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
14          on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
15          and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
16          otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
17          A non-weights segmenter returns True (nothing local to check)."""

```

```

18     name = (seg_name or "").strip().lower()
19     if name not in _WEIGHTS_BACKED_SEGMENTERS:
20         return True
21     idx = getattr(cfg, "indexing", None)
22     substrate = (
23         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
24     ).strip().lower()
25     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
26         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
27     else: # wasb_hybrid, or fusion on the (default) WASB substrate
28         path = (
29             os.environ.get("WASB_WEIGHTS")
30             or getattr(getattr(idx, "wasb", None), "weights_path", "")
31             or ""
32         )
33     return bool(path) and os.path.exists(os.path.expanduser(path))
34
35
36 def _cloud_available(cfg) -> bool:
37     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
38 ).
39     True when the control plane is wired for Cloud Run: either the configured default
40     target is
41     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
42     coordinates
43     (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
44     (``storage.output_root``)
45     are present so a forced per-request override can actually dispatch + collect a
46     result.
47
48     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
49     would work."""
50     deployment = getattr(cfg, "deployment", None)
51     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
52         return True
53     has_job = bool(
54         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
55     )
56     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
57     return bool(has_job and out_root)
58
59
60 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
61     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
62     minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
63     comes
64     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
65     "starting"
66     (queue/cold-start), so the very first tick already shows movement."""
67     minutes = int(elapsed_sec) // 60
68     if minutes < 1:
69         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
70     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
71
72
73 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
74     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
75     gs://
76     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
77     keep
78     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
79     no
80     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""

```

```

71     if getattr(input_ref, "backend", "") != "gcs":
72         return None
73     sc = _storage_settings(cfg)
74     if not sc:
75         return None
76     try:
77         st = storage.get_storage("gcs", config=sc).stat(input_ref)
78         return getattr(st, "md5", None) or None
79     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
80         logger.info(
81             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
82             getattr(input_ref, "uri", input_ref),
83             exc_info=True,
84         )
85     return None
86
87
88     def _cached_process_payload(
89         video_id: str, validation_results, segments, requested_compute
90     ) -> Dict[str, Any]:
91         """The /api/process result for a video whose detection already exists ? ONE shape
92         for
93         both producers (the drain's DB-cache short-circuit and the submit-time cache,
94         #484)."""
95         return {
96             "video_id": video_id,
97             "status": "success",
98             "message": (
99                 f"Reused {len(segments)} cached play segments. "
100                 "Pass force=true to reprocess this video."
101             ),
102             "results": validation_results or [],
103             "play_segments": segments,
104             "cached": True,
105             "compute": {"path": "cached", "requested": requested_compute},
106         }
107
108     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
109         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
110         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
111         decide."""
112         ref = storage.parse_uri(uri)
113         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
114             local = storage.fetch(
115                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
116             )
117             with open(local, encoding="utf-8") as f:
118                 loaded = json.load(f)
119             return loaded if isinstance(loaded, dict) else None
120
121     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
122     hits
123     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
124     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
125     instance
126     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
127     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
128     _REHYDRATE_LOCKS_GUARD = threading.Lock()
129
130     def _rehydrate_lock(video_id: str) -> threading.Lock:
131         with _REHYDRATE_LOCKS_GUARD:
132             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())

```

```

131
132
133     def probe_process_cache(db, cfg, video_id: str) -> bool:
134         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
135
136         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
137         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
138         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
139         errors report False (? the normal dispatch path with its guards)."""
140         try:
141             existing = db.get_video(video_id)
142             if (
143                 existing
144                 and existing.get("status") == "processed"
145                 and db.query_play_segments(video_id, {})
146             ):
147                 return True
148             sc = _storage_settings(cfg)
149             if not sc or (sc.get("backend") or "local") == "local":
150                 return False
151             output_root = (sc.get("output_root") or "").rstrip("/")
152             if not output_root:
153                 return False
154             report_uri = f"{output_root}/outputs/{video_id}/report.json"
155             if not storage.exists(report_uri, config=sc):
156                 return False
157             report = _read_bucket_json(report_uri, sc)
158             return bool(report and report.get("status") == "success")
159         except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
160             logger.warning(
161                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
162             )
163         return False
164
165
166     def cached_process_result(
167         db, cfg, video_id: str, video_path: str, requested_compute=None
168     ) -> Optional[Dict[str, Any]]:
169         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
170         the DB first, then the bucket (``outputs/<md>report.json`` with ``status=success``
171         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
172         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
173         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
174         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
175         partial earlier ingest is replaced, never double-counted.
176
177         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
178         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
179         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
180         job before invoking (writes happen post-claim)."""
181         try:
182             with _rehydrate_lock(video_id):
183                 existing = db.get_video(video_id)
184                 if existing and existing.get("status") == "processed":
185                     segs = db.query_play_segments(video_id, {})

```

```

186         if segs:
187             return _cached_process_payload(
188                 video_id,
189                 existing.get("validation_results", []),
190                 segs,
191                 requested_compute,
192             )
193     sc = _storage_settings(cfg)
194     if not sc or (sc.get("backend") or "local") == "local":
195         return None
196     output_root = (sc.get("output_root") or "").rstrip("/")
197     if not output_root:
198         return None
199     outputs_uri = f"{output_root}/outputs/{video_id}"
200     report_uri = f"{outputs_uri}/report.json"
201     if not storage.exists(report_uri, config=sc):
202         return None
203     report = _read_bucket_json(report_uri, sc)
204     if not report or report.get("status") != "success":
205         return None
206     if not video_path:
207         # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
208         # carries the worker's --video-uri (P3); older reports fall back to the
209         # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
210         # recover, and the next same-md5 submit re-binds the truthful path via
211         # add_video's upsert.
212         video_path = str(report.get("video_uri") or "") or _first_object_under(
213             f"{output_root}/videos/{video_id}/", sc
214         )
215     play_wm, cand_wm = db.segment_watermarks(video_id)
216     db.add_video(video_id, video_path, "processed", [])
217     n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
218     if n <= 0:
219         return None
220     segments = [
221         s
222         for s in db.query_play_segments(video_id, {})
223         if int(s.get("id", 0)) > play_wm
224     ]
225     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
226     if not segments:
227         return None
228     logger.info(
229         "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
230         n,
231         video_id,
232         outputs_uri,
233     )
234     return _cached_process_payload(video_id, [], segments, requested_compute)
235 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
236     logger.warning(
237         "[API] submit-time cache check failed for %s ? dispatching normally",
238         video_id,
239         exc_info=True,
240     )
241     return None
242
243
244 def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
245     """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
246     try:
247         return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")

```

```

248     except Exception: # noqa: BLE001 ? recovery nicety, never a failure
249         return ""
250
251
252     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
253
254
255     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
256         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
257         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
258         so
259         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
260         DB
261         has the video's rows now (re-query). Non-md5 ids return False without touching the
262         bucket; every failure degrades to False (the read just reports what the DB has)."""
263         vid = (video_id or "").strip().lower()
264         if not _MD5_HEX_RE.fullmatch(vid):
265             return False
266         if not probe_process_cache(db, cfg, vid):
267             return False
268         return cached_process_result(db, cfg, vid, "") is not None
269
270     def _maybe_dispatch_remote(
271         cfg,
272         db,
273         req: "ProcessRequest",
274         video_id: str,
275         seg_name: str,
276         val_results,
277         play_wm: int,
278         cand_wm: int,
279         notify,
280     ) -> Optional[Tuple[Dict[str, Any], bool]]:
281         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
282         ``deployment.compute_target
283         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
284         intervals into
285         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
286         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
287         the cloud job
288         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
289         observability report
290         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
291         process segmenter
292         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
293         non-``cloud_run`` target, so
294         the in-process path stays byte-identical)."""
295         from backend.compute import ComputeJobSpec, get_compute_backend
296
297         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
298             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
299             (id > play_wm);
300             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
301             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
302             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
303             only when the
304             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
305             rc / ingest
306             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
307             records the
308             # intended backend, `requested` the picker choice, `dispatched` whether the
309             remote job ran.
310             return (

```

```

300         {
301             "video_id": video_id,
302             "status": "failed",
303             "message": msg,
304             "results": [r.model_dump() for r in val_results],
305             "play_segments": [],
306             "compute": {
307                 "path": "cloud_run" if ran else "not_run",
308                 "requested": req.compute,
309                 "target": "cloud_run",
310                 "dispatched": ran,
311             },
312         },
313         ran,
314     )
315
316     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
317     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
318     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
319     choice = (req.compute or "").strip().lower()
320     if choice and choice not in ("local", "cloud"):
321         logger.warning(
322             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
323             req.compute,
324         )
325     choice = ""
326     if choice == "local":
327         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
328         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
329         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
330         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
331         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
332         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
333         # below exactly as before (the weights-present check is the escape hatch).
334         if (
335             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
336             and _cloud_available(cfg)
337             and not _local_detector_weights_present(cfg, seg_name)
338         ):
339             return _failed(
340                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
341                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
342                 False,
343             )
344         return None # explicit "run on my machine" ? in-process regardless of the
server's target
345     if choice == "cloud":
346         if not _cloud_available(cfg):
347             return _failed(
348                 "cloud-serving was requested but this server isn't configured for it "
349                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
350                 False,

```

```

351         )
352         compute = get_compute_backend(cfg, target_override="cloud_run")
353     else:
354         compute = get_compute_backend(
355             cfg
356         ) # server default (default-OFF unless cloud_run)
357     if compute is None:
358         return None # default-OFF ? run the in-process segmenter below (unchanged)
359
360     storage_cfg = getattr(cfg, "storage", None)
361
362     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
363     # to this box.
364     try:
365         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
366     except ValueError as e:
367         return _failed(

```

11. user (2026-07-07T15:09:33.115Z)

```

=== repo root ===
__pycache__
artifacts
backend
backend.db
CLAUDE.md
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
README.md
requirements.txt
requirements-trackers.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
score_golden_gate.py
score_local.py
scratch
scripts
SESSION_HANDOFF.md
tests
tools
training
=== find key files ===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/tracknet_runner.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/segmenters/fusion_hybrid.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/segmenters/tracknet_hybrid.py

```



```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-  
indexer/backend/pipeline/segmenters/trajectory_hybrid.py  
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-  
indexer/backend/pipeline/segmenters/wasb_hybrid.py
```

12. assistant (2026-07-07T15:09:48.064Z)

Now let me read the actual runner, segmenter, config, and capabilities-probe files for ground-truth weights resolution.

13. user (2026-07-07T15:09:49.063Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).  
2  
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?  
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``  
M3.5).  
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a  
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``  
core  
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float  
tolerance;  
8      see the acceptance gate below) ? but WITHOUT the WSL transport:  
9  
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the  
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation  
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.  
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or  
``auto``  
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit  
``cuda`` stays  
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra  
``runner.device`` override).  
16     * weights / repo / python / scratch come from the environment first  
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then  
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.  
19  
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``  
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /  
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and  
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam  
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);  
``detector_impl="native``".  
25  
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL  
runner  
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the  
same  
28     candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-  
deterministic  
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-  
exact  
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native  
streaming  
31     vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge  
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware  
check  
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the  
contract,  
34     not the numbers.  
35     ""  
36  
37     import logging  
38     import os  
39     import shutil
```

```

40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67     # Probe for the GPU-resident decode dep (#506): a wasb env without PyNvVideoCodec must
fail the
68     # healthcheck, not the paid inference subprocess. (The NVDEC driver libs ? libnvcuvid ?
are only
69     # exercised at decoder INIT, so a missing lib still surfaces in the subprocess; this
catches the
70     # common misconfig, a wasb env built without the package.)
71     _PYNVC_PROBE_CODE = "import PyNvVideoCodec"
72
73
74     @dataclass
75     class NativeWasbConfig:
76         """Resolved settings for a Linux-native WASB run.
77
78         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
79         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
80         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
81         """
82
83         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
84         python_bin: str = (
85             "python" # the `wasb` conda env python (invoked by path, no activate)
86         )
87         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
88         sport: str = "badminton"
89         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
90         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
91         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
92         keep_frames: bool = (
93             False # retain the decoded frame cache after success (debug only)
94         )
95         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long

```

```

clip's
96      # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
97      # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
98      stream_video: bool = True
99      # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
100     # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
101     batch_size: int = 0
102     prefetch_batches: int = 0
103     # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
104     # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
105     # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
106     decode_backend: str = "cpu"
107
108     @classmethod
109     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
110         if isinstance(idx_cfg, dict):
111             idx_cfg = IndexingConfig(**idx_cfg)
112         w = idx_cfg.wasb
113         env = os.environ.get
114         # stream_video is tri-state in config: None/absent => native default (stream);
an
115         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
116         sv = w.stream_video
117         return cls(
118             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
119             python_bin=env("WASB_PYTHON") or w.python_bin,
120             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
121             sport=w.sport,
122             device=env("WASB_DEVICE") or w.device,
123             scratch_dir=env("WASB_SCRATCH")
124             or env("RALLY_SCRATCH")
125             or env("TMPDIR")
126             or "",
127             timeout_sec=w.timeout_sec,
128             keep_frames=w.keep_frames,
129             stream_video=(True if sv is None else bool(sv)),
130             batch_size=int(w.batch_size or 0),
131             prefetch_batches=int(w.prefetch_batches or 0),
132             decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
133         )
134
135
136     class NativeWasbRunner(DetectorRunner):
137         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
138
139         def __init__(self, cfg: NativeWasbConfig):
140             self.cfg = cfg
141             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
142             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
143             self._cuda_available: Optional[bool] = None
144
145             # --- low-level native exec (the single place tests patch)
-----
146             def _run(
147                 self, argv: List[str], cwd: Optional[str] = None
148             ) -> subprocess.CompletedProcess:

```

```

149         logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
150         return subprocess.run(
151             argv,
152             cwd=cwd,
153             capture_output=True,
154             text=True,
155             timeout=(self.cfg.timeout_sec or None),
156         )
157
158     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
159     def _probe_cuda(self) -> bool:
160         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
161         (missing python / timeout / import error) is treated as 'no CUDA'."""
162         try:
163             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
164         except (FileNotFoundError, subprocess.TimeoutExpired):
165             return False
166         return res.returncode == 0 and "cuda True" in (res.stdout or "")
167
168     def _cuda_is_available(self) -> bool:
169         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
170         if self._cuda_available is None:
171             self._cuda_available = self._probe_cuda()
172         return self._cuda_available
173
174     def _effective_device(self) -> str:
175         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
176         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
177         if self.cfg.device == AUTO:
178             return DeviceContext(AUTO, self._cuda_is_available()).effective
179         return self.cfg.device
180
181     def _repo_src(self) -> str:
182         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
183
184     def _copy_infer_script(self) -> bool:
185         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
186         repo_src = self._repo_src()
187         try:
188             os.makedirs(repo_src, exist_ok=True)
189             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
190             return True
191         except OSError as e:
192             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
193             return False
194
195     def _rm_rf(self, path: Optional[str]) -> None:
196         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
197         if path:
198             shutil.rmtree(path, ignore_errors=True)
199
200     def healthcheck(self) -> Tuple[bool, str]:
201         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
202         if not self.cfg.weights_path:
203             return False, (
204                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."

```

```

205         )
206         weights = os.path.expanduser(self.cfg.weights_path)
207         if not os.path.exists(weights):
208             return False, f"WASB weights not found at {weights}"
209         repo_src = self._repo_src()
210         if not os.path.isdir(repo_src):
211             return False, (
212                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
213                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
214         )
215         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
216         # argparse rc=2 from wasb_infer.py at run time.
217         if self.cfg.device not in VALID_DEVICES:
218             return False, (
219                 f"unknown device {self.cfg.device!r} ? valid: {'',
220                 '.join(VALID_DEVICES)})."
221             )
222         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
223             return False, (
224                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
225                 f"{'', '.join(_VALID_DECODE_BACKENDS)})."
226             )
227         try:
228             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
229         except FileNotFoundError:
230             return (
231                 False,
232                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
233             )
234         except subprocess.TimeoutExpired:
235             return False, "native torch healthcheck timed out."
236         out = (res.stdout or "").strip()
237         if res.returncode != 0:
238             return (
239                 False,
240                 f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
241                 out).strip()}",
242             )
243         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
244         subprocess).
245         self._cuda_available = "cuda True" in out
246         ctx = DeviceContext(self.cfg.device, self._cuda_available)
247         if ctx.cuda_required_but_missing:
248             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
249             a silent
250             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
251             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
252         if ctx.fell_back:
253             logger.warning(
254                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
255                 "inference will be slow. Set indexing.wasb.device=cuda to require a
256                 GPU.",
257                 out,
258             )
259         # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
260         the
261         # config validator already rejects) cannot run the GPU-resident path ? fail loud
262         # instead of letting wasb_infer.py die mid-run on a paid box.
263         if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
264             return False, (
265                 f"decode_backend=nvdec_resident requires CUDA, but the effective device
266                 is "
267                 f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
268             )

```

```

262         if self.cfg.decode_backend == "nvdec_resident":
263             try:
264                 nv = self._run([self.cfg.python_bin, "-c", _PYNVC_PROBE_CODE])
265             except (FileNotFoundError, subprocess.TimeoutExpired):
266                 return False, "PyNvVideoCodec probe failed to run (python
missing/timeout)."
267             if nv.returncode != 0:
268                 return False, (
269                     f"decode_backend=nvdec_resident but `{_PYNVC_PROBE_CODE}` failed in
"
270                     f"{self.cfg.python_bin!r}: {(nv.stderr or nv.stdout or
''.strip())[-300:]} "
271                     f"? the wasb env needs PyNvVideoCodec (deploy/cloudrun/Dockerfile /
"
272                     f"gpu_setup.sh stage it).")
273             )
274         return True, out
275
276     def run_predict(
277         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
278     ) -> Optional[str]:
279         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
280
281         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
282         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
283         same downstream contract, so the parity comparison is apples-to-apples.
284         """
285         output_dir = os.path.abspath(output_win_dir)
286         os.makedirs(output_dir, exist_ok=True)
287         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
288         norm = str(video_win_path).replace("\\", "/")
289         stem, _ext = os.path.splitext(os.path.basename(norm))
290         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
291         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
292         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
293
294         if not self._copy_infer_script():
295             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
296             return None
297
298         weights = os.path.expanduser(self.cfg.weights_path)
299         argv = [
300             self.cfg.python_bin,
301             "wasb_infer.py",
302             "--video",
303             os.path.abspath(str(video_win_path)),
304         ]
305         frames_dir: Optional[str] = None
306         if self.cfg.decode_backend == "nvdec_resident":
307             # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
itself
308             # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
309             # --stream-video nor --frames_out_dir applies and there is no frame cache to
clean.
310             argv += ["--decode-backend", self.cfg.decode_backend]
311         elif self.cfg.stream_video:
312             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
313             argv.append("--stream-video")
314         else:

```

```

315         scratch = self.cfg.scratch_dir or output_dir
316         frames_dir = os.path.join(scratch, f"{key}_frames")
317         argv += ["--frames_out_dir", frames_dir]
318     # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
319     # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
320     device = self._effective_device()
321     if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
322         # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
a
323         # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
324         logger.error(
325             "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
326             "%r ? aborting before the WASB subprocess.",
327             device,
328         )
329         return None
330     argv += [
331         "--weights",
332         weights,
333         "--sport",
334         self.cfg.sport,
335         "--device",
336         device,
337         "--out",
338         out_csv,
339     ]
340     # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
341     # byte-parity guarantee) is unchanged when the knobs are unset.
342     if self.cfg.batch_size:
343         argv += ["--batch-size", str(self.cfg.batch_size)]
344     if self.cfg.prefetch_batches and self.cfg.decode_backend != "nvdec_resident":
345         # Prefetch overlaps CPU-side batch assembly with GPU compute ? meaningless
for
346         # the GPU-resident decoder (there is no CPU assembly), whose wasb_infer
branch
347         # ignores the flag; keep the argv honest instead of passing a dead knob.
348         argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
349
350     logger.info(
351         "Running native WASB inference (device=%s%s) on %s ...",
352         device,
353         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
354         os.path.basename(norm),
355     )
356     try:
357         res = self._run(argv, cwd=self._repo_src())
358     except FileNotFoundError:
359         logger.error(
360             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
361         )
362         return None
363     except subprocess.TimeoutExpired:
364         logger.error(
365             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
366         )
367         return None
368     if res.returncode != 0:
369         logger.error(
370             "WASB native inference failed:\n%s",
371             (res.stderr or res.stdout or "")[-2000:],

```

```

372         )
373         return None
374     if not os.path.exists(out_csv):
375         logger.error(
376             "WASB native finished but expected CSV not found at %s", out_csv
377         )
378         return None
379     logger.info("WASB trajectory CSV: %s", out_csv)
380
381     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
382     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
383     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
384     if not self.cfg.keep_frames and frames_dir:
385         logger.info(
386             "Cache hygiene: removing frame cache for %s "
387             "(set indexing.wasb.keep_frames=true to retain).",
388             key,
389         )
390         self._rm_rf(frames_dir)
391     return out_csv
392
393     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
394     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
395     trajectory_to_action_windows = staticmethod(
396         TrackNetRunner.trajectory_to_action_windows
397     )
398

```

14. user (2026-07-07T15:09:50.000Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12
13    import logging
14    import os
15    import shlex
16    import subprocess
17    from dataclasses import dataclass
18    from typing import Optional, Tuple
19
20    from backend.config.models import IndexingConfig
21    from backend.pipeline.detectors.base import (
22        DetectorRunner,
23        WslCommandMixin,
24        wsl_tilde_quote,
25    )
26
27    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
28    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
29    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, to_wsl_mnt_path
30

```



```

31     logger = logging.getLogger(__name__)
32
33     # Windows path to the inference wrapper we stage into WSL.
34     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
35
36
37     @dataclass
38     class WasbConfig:
39         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
40
41         distro: str = "Ubuntu"
42         repo_dir: str = "~/models/WASB-SBDT"
43         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
44         conda_env: str = "wasb"
45         weights_path: str = (
46             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
47         )
48         sport: str = "badminton"
49         wsl_stage_dir: str = "~/clips"
50         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
51         keep_frames: bool = (
52             False # retain staged video + frame cache after success (debug only)
53         )
54         min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
= off)
55
56         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
57         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
58         # extraction that dominates a long clip. Output is bit-identical to the PNG path
59         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
60         it.
61
62         stream_video: bool = False
63
64
65     @classmethod
66     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "WasbConfig":
67         if isinstance(idx_cfg, dict):
68             idx_cfg = IndexingConfig(**idx_cfg)
69         w = idx_cfg.wasb
70         return cls(
71             distro=w.wsl_distro,
72             repo_dir=w.repo_dir,
73             conda_sh=w.conda_sh,
74             conda_env=w.conda_env,
75             weights_path=w.weights_path,
76             sport=w.sport,
77             wsl_stage_dir=w.wsl_stage_dir,
78             timeout_sec=w.timeout_sec,
79             keep_frames=w.keep_frames,
80             min_free_gb=w.min_free_gb,
81             stream_video=w.stream_video if w.stream_video is not None else False,
82         )
83
84
85     class WasbRunner(WslCommandMixin, DetectorRunner):
86         def __init__(self, cfg: WasbConfig):
87             self.cfg = cfg
88
89
90         def healthcheck(self) -> Tuple[bool, str]:
91             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
92             cmd = (
93                 f"conda activate {self.cfg.conda_env} && "
94                 f"python -c \"import torch; print('torch', torch.__version__, "
95                 f"'cuda', torch.cuda.is_available())\""
96             )
97             try:

```

```

93         res = self._wsl_bash(cmd)
94     except FileNotFoundError:
95         return False, "`wsl` not found ? Windows + WSL2 required."
96     except subprocess.TimeoutExpired:
97         return False, "WSL healthcheck timed out."
98     out = (res.stdout or "").strip()
99     if res.returncode != 0:
100         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
101     return True, out
102
103 def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
104     src_mnt = to_wsl_mnt_path(video_win_path)
105     reason = self.wsl_source_unreadable_reason(src_mnt)
106     if reason:
107         logger.error("Cannot stage video into WSL: %s", reason)
108         return None
109     dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
110     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
111     {wsl_tilde_quote(dest)} && echo OK"
112     try:
113         res = self._wsl_bash(cmd)
114     except subprocess.TimeoutExpired:
115         logger.error("Staging video into WSL timed out.")
116         return None
117     if res.returncode != 0 or "OK" not in (res.stdout or ""):
118         logger.error(
119             "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
120         )
121         return None
122     return dest
123
124 def _stage_infer_script(self) -> bool:
125     """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
126     src_mnt = to_wsl_mnt_path(_INFER_WIN)
127     cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo"
128     OK"
129     res = self._wsl_bash(cmd)
130     return res.returncode == 0 and "OK" in (res.stdout or "")
131
132 def run_predict(
133     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
134 ) -> Optional[str]:
135     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
136
137     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
138     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
139     can never reuse each other's staged frames, resumable cache, or CSV.
140
141     """
142     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
143     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
144     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
145     # the CSV landed inside WSL while Windows polled the repo-relative path.
146     output_win_dir = os.path.abspath(output_win_dir)
147     os.makedirs(output_win_dir, exist_ok=True)
148     # Normalize for cross-platform basename (tests use Windows paths on Linux).
149     video_win_path = str(video_win_path).replace("\\", "/")
150     base = os.path.basename(video_win_path)
151     stem, ext = os.path.splitext(base)
152     # Unique, content-derived key: same filename + different bytes -> different key.
153     key = f"{stem}__{video_id[:12]}" if video_id else stem
154     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
155     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
156
157     if not self._stage_infer_script():
158         logger.error("Could not stage wasb_infer.py into WSL.")

```

```

156         return None
157     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
158     if staged_video is None:
159         return None
160     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
161
162     # Disk guard (D11): a 4K clip can extract ~100 GB of PNG frames. BLOCK before
extraction if
163     # the WSL stage fs can't hold ~15× the source ? a mid-run disk-fill wastes a
long run and
164     # looks like a quality fail. Best-effort measuring (an undeterminable read never
blocks).
165     # Streaming never materializes the PNGs, so the guard only matters off the fast
path.
166     if not self.cfg.stream_video:
167         shortfall = self._wsl_frame_stage_shortfall(video_win_path)
168         if shortfall:
169             logger.error("WASB inference aborted ? %s", shortfall)
170             return None
171
172     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
173     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
174     if self.cfg.stream_video:
175         frame_args = "--stream-video "
176     else:
177         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
178     cmd = (
179         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
180         f"python wasb_infer.py "
181         f"--video {wsl_tilde_quote(staged_video)} "
182         f"{frame_args}"
183         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
184         f"--sport {shlex.quote(self.cfg.sport)} "
185         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
186     )
187     logger.info("Running WASB inference on %s ...", base)
188     try:
189         res = self._wsl_bash(cmd)
190     except subprocess.TimeoutExpired:
191         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
192         return None
193     if res.returncode != 0:
194         logger.error(
195             "WASB inference failed:\n%s", (res.stderr or res.stdout)[-2000:]
196         )
197         return None
198     if not os.path.exists(expected_csv_win):
199         logger.error(
200             "WASB finished but expected CSV not found at %s", expected_csv_win
201         )
202         return None
203     logger.info("WASB trajectory CSV: %s", expected_csv_win)
204
205     # Success ? the trajectory CSV is the durable output; the staged video copy and
206     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
207     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
208     # so the frame cache + detector cache remain for a resumable re-run.
209     # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
210     if not self.cfg.keep_frames:
211         stale = (
212             [staged_video] if self.cfg.stream_video else [frames_dir, staged_video]
213         )
214         logger.info(

```

```

215         "Cache hygiene: removing staged video%s for %s "
216         "(set indexing.wasb.keep_frames=true to retain).",
217         "" if self.cfg.stream_video else " + frame cache",
218         key,
219     )
220     self._wsl_rm_rf(stale)
221     return expected_csv_win
222
223     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
224     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
225     trajectory_to_action_windows = staticmethod(
226         TrackNetRunner.trajectory_to_action_windows
227     )
228

```

15. user (2026-07-07T15:09:50.747Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import csv
18     import logging
19     import os
20     import shlex
21     import subprocess
22     from dataclasses import dataclass
23     from typing import Any, Dict, List, Optional, Tuple
24
25     from backend.config.models import IndexingConfig
26     from backend.pipeline.detectors.base import (
27         DetectorRunner,
28         WslCommandMixin,
29         wsl_tilde_quote,
30     )
31
32     logger = logging.getLogger(__name__)
33
34
35     @dataclass
36     class TrackNetConfig:
37         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
38
39         distro: str = "Ubuntu"
40         repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
41         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
42         conda_env: str = "TrackNetV4"
43         weights_path: str = "" # WSL path to the .h5/.keras weights (REQUIRED)
44         queue_length: int = 5
45         stage_in_wsl: bool = True # copy video into WSL fs first (perf)
46         wsl_stage_dir: str = "~/clips" # where staged videos/outputs live in WSL

```

```

47     timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no timeout)
48     keep_frames: bool = False # retain staged video after success (debug only)
49     min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
= off)
50
51     @classmethod
52     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "TrackNetConfig":
53         if isinstance(idx_cfg, dict):
54             idx_cfg = IndexingConfig(**idx_cfg)
55         tn = idx_cfg.tracknet
56         return cls(
57             distro=tn.wsl_distro,
58             repo_dir=tn.repo_dir,
59             conda_sh=tn.conda_sh,
60             conda_env=tn.conda_env,
61             weights_path=tn.weights_path,
62             queue_length=tn.queue_length,
63             stage_in_wsl=tn.stage_in_wsl,
64             wsl_stage_dir=tn.wsl_stage_dir,
65             timeout_sec=tn.timeout_sec,
66             keep_frames=tn.keep_frames,
67             min_free_gb=tn.min_free_gb,
68         )
69
70
71     def to_wsl_mnt_path(win_path: str) -> str:
72         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
73
74         Already-POSIX paths (starting with / or ~) are returned unchanged so the
75         same helper is safe to call on either kind of path.
76         """
77         p = str(win_path)
78         if p.startswith("/") or p.startswith("~"):
79             return p
80         p = p.replace("\\", "/")
81         if len(p) >= 2 and p[1] == ":":
82             drive = p[0].lower()
83             return f"/mnt/{drive}{p[2:]}"
84         return p
85
86
87     @dataclass
88     class TrajectoryPoint:
89         frame: int
90         visible: bool
91         x: float
92         y: float
93
94
95     class TrackNetRunner(WslCommandMixin, DetectorRunner):
96         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
97
98         Implements the common DetectorRunner contract so a future Linux-native or
99         alternative trajectory runner can be substituted without touching the hybrids.
100         """
101
102         def __init__(self, cfg: TrackNetConfig):
103             self.cfg = cfg
104
105         def healthcheck(self) -> Tuple[bool, str]:
106             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108             Returns (ok, message). Cheap to call before a long run.
109             """
110             cmd = (

```

```

111         f"conda activate {self.cfg.conda_env} && "
112         f'python -c "import tensorflow as tf; '
113         f'print(\'TF\', tf.__version__, \'GPUs\', '
len(tf.config.list_physical_devices(\'GPU\')))\\"
114     )
115     try:
116         res = self._wsl_bash(cmd)
117     except FileNotFoundError:
118         return (
119             False,
120             "\`wsl\` not found ? is this running on Windows with WSL2 installed?",
121         )
122     except subprocess.TimeoutExpired:
123         return False, "WSL healthcheck timed out."
124     out = (res.stdout or "").strip()
125     if res.returncode != 0:
126         return False, f"WSL env check failed: {(res.stderr or out).strip()}"
127     return True, out
128
129 # ----- inference
130
131 def run_predict(
132     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
133 ) -> Optional[str]:
134     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
135
136     ``output_win_dir`` is a Windows directory (under the repo's output/) that
137     WSL writes into via its /mnt view, so the Windows process can read the CSV
138     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
139     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
140     a filename cannot collide on the output CSV.
141     """
142     if not self.cfg.weights_path:
143         logger.error(
144             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
145             "in config.json to the WSL path of your trained TrackNetV4 weights."
146         )
147     return None
148
149     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
150     # relative paths through unchanged, so WSL would resolve them against the
151     # predict.py cwd instead of the repo (same bug class as wasb_runner).
152     output_win_dir = os.path.abspath(output_win_dir)
153     os.makedirs(output_win_dir, exist_ok=True)
154     # Normalize separators so basename/splitext work when tests (or collector)
155     # pass Windows-style paths on a Linux host.
156     video_win_path = str(video_win_path).replace("\\", "/")
157     base = os.path.basename(video_win_path)
158     stem, ext = os.path.splitext(base)
159
160     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
161     if ext.lower() not in (".mp4", ".avi"):
162         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
163     return None
164
165     wsl_out = to_wsl_mnt_path(output_win_dir)
166     # Namespace by video_id so two same-filename videos don't collide on the CSV.
167     # predict.py derives its output name from the (staged) video stem, so staging
168     # under the namespaced name propagates into "<key>_predict.csv".
169     key = f"{stem}__{video_id[:12]}" if video_id else stem
170     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
171
172     # Resolve the video path WSL will read.
173     if self.cfg.stage_in_wsl:

```

```

174         staged = self._stage_video(video_win_path, f"{key}{ext}")
175         if staged is None:
176             return None
177         wsl_video = staged
178     else:
179         # No staging: predict.py reads /mnt directly and writes
180         # We cannot rename its output, so fall back to the un-namespaced name.
181         wsl_video = to_wsl_mnt_path(video_win_path)
182         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
183
184         # Disk guard (D11): BLOCK before extraction if the WSL stage fs can't hold ~15x
185         # (frame PNGs balloon the footprint). Best-effort measuring ? an undeterminable
186         # blocks; min_free_gb (config) still acts as an absolute floor. See
187         # base._wsl_frame_stage_shortfall.
188         shortfall = self._wsl_frame_stage_shortfall(video_win_path)
189         if shortfall:
190             logger.error("TrackNet inference aborted ? %s", shortfall)
191             return None
192
193         cmd = (
194             f"conda activate {self.cfg.conda_env} && "
195             f"cd {self.cfg.repo_dir}/src && "
196             f"python predict.py "
197             f"--video_path {wsl_tilde_quote(wsl_video)} "
198             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
199             f"--output_dir {wsl_tilde_quote(wsl_out)} "
200             f"--queue_length {self.cfg.queue_length}"
201         )
202         logger.info("Running TrackNet inference on %s ...", base)
203         try:
204             res = self._wsl_bash(cmd)
205         except subprocess.TimeoutExpired:
206             logger.error(
207                 "TrackNet inference timed out after %ss.", self.cfg.timeout_sec
208             )
209             return None
210
211         if res.returncode != 0:
212             logger.error(
213                 "TrackNet predict.py failed:\n%s", (res.stderr or res.stdout)[-2000:]
214             )
215             return None
216
217         if not os.path.exists(expected_csv_win):
218             logger.error(
219                 "TrackNet finished but expected CSV not found at %s", expected_csv_win
220             )
221             return None
222         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
223
224         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
225         # regenerable intermediate). On earlier failure we return above without
226         # cleaning.
227         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
228             logger.info(
229                 "Cache hygiene: removing staged video for %s "
230                 "(set indexing.tracknet.keep_frames=true to retain).",
231                 key,
232             )
233             self._wsl_rm_rf([wsl_video])
234         return expected_csv_win

```

```

234 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
235     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
236
237     Returns the WSL path to the staged copy, or None on failure.
238     """
239     src_mnt = to_wsl_mnt_path(video_win_path)
240     reason = self.wsl_source_unreadable_reason(src_mnt)
241     if reason:
242         logger.error("Cannot stage video into WSL: %s", reason)
243         return None
244     dest = f"{self.cfg.wsl_stage_dir}/{base}"
245     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
246     try:
247         res = self._wsl_bash(cmd)
248     except subprocess.TimeoutExpired:
249         logger.error("Staging video into WSL timed out.")
250         return None
251     if res.returncode != 0 or "OK" not in (res.stdout or ""):
252         logger.error(
253             "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
254         )
255         return None
256     return dest
257
258 # ----- CSV -> windows
259
260 @staticmethod
261 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
262     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
263     points: List[TrajectoryPoint] = []
264     with open(csv_win_path, newline="") as f:
265         reader = csv.DictReader(f)
266         for row in reader:
267             try:
268                 points.append(
269                     TrajectoryPoint(
270                         frame=int(row["Frame"]),
271                         visible=int(row["Visibility"]) == 1,
272                         x=float(row["X"]),
273                         y=float(row["Y"]),
274                     )
275             )
276         except (KeyError, ValueError):
277             continue
278     points.sort(key=lambda p: p.frame)
279     return points
280
281 @staticmethod
282 def _active_frames(
283     points: List[TrajectoryPoint],
284     fps: float,
285     frame_width: float,
286     velocity_thresh: float,
287 ) -> List[Tuple[int, float]]:
288     """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
289
290     Computes each visible point's normalised velocity from the last *visible* point;
291     an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
292     ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
293     the caller so identical floats feed the velocity math."""
294     active = [] # (frame_idx, velocity)

```



```

295     prev: Optional[TrajectoryPoint] = None
296     for p in points:
297         if not p.visible:
298             prev = None # break the trajectory; absent shuttle = not in flight
299             continue
300         if prev is not None:
301             dframes = p.frame - prev.frame
302             if dframes > 0:
303                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
304                 velocity = (dist / frame_width) * (fps / dframes)
305                 if velocity > velocity_thresh:
306                     active.append((p.frame, velocity))
307             prev = p
308     return active
309
310 @staticmethod
311 def trajectory_to_action_windows(
312     points: List[TrajectoryPoint],
313     fps: float,
314     frame_width: float,
315     chunk_start: float = 0.0,
316     velocity_thresh: float = 0.02,
317     merge_gap: float = 2.0,
318     min_window_duration: float = 2.0,
319     chunk_index: Optional[int] = None,
320     max_window_duration: float = 0.0,
321     min_split_gap: float = 0.5,
322     window_hard_cap: bool = False,
323     window_inactivity_end: float = 0.0,
324     inactivity_min_density: float = 0.34,
325     inactivity_rally_thresh: float = 0.08,
326     inactivity_temporal_density: bool = False,
327     window_blob_fallback: bool = False,
328     window_blob_factor: float = 4.0,
329 ) -> List[Dict[str, Any]]:
330     """Convert a shuttle trajectory into candidate rally windows.
331
332     A frame is "active" when the shuttle is visible AND its inter-frame
333     displacement (normalised by frame width, per second) exceeds
334     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
335     ``merge_gap`` seconds of each other are grouped into one window; short
336     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
337     shape feeding the AI-handoff phase is identical.
338
339     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
340     against the *over-merge* failure where "shuttle moving" is conflated with "rally in
341     progress" and a single window swallows several rallies + the dead time between them. When
342     set, any window longer than this is recursively SPLIT at its largest internal inactivity
343     gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
344     provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
345     fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
346     recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
347
348     ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
349     an over-long window has NO internal inactivity gap to split on (a continuously-
350     active trajectory ? the shuttle never stops moving, e.g. the real-footage

```

```

``mahadevpura-1``
351         blob where the gap-splitter alone left a single 174 s window), it is recursively
352         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
353         (never merges), so recall cannot drop; gated default-OFF until measured.
354
355         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
356         ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
357         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
358         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` *
359         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
360         their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
361         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
362         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
363         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
364         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
365         ""
366         if fps <= 0:
367             fps = 30.0
368         if frame_width <= 0:
369             frame_width = 1280.0
370
371         active = TrackNetRunner._active_frames(
372             points, fps, frame_width, velocity_thresh
373         )
374
375         if not active:
376             return []
377
378         max_gap_frames = int(merge_gap * fps)
379
380         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
381             vels = [v for _, v in group]
382             return {
383                 "start": group[0][0] / fps + chunk_start,
384                 "end": group[-1][0] / fps + chunk_start,
385                 "active_frame_count": len(group),
386                 "peak_velocity": max(vels),
387                 "mean_velocity": sum(vels) / len(vels),
388                 "track_id_count": 1, # single shuttle; kept for window-shape parity
389                 "chunk_index": chunk_index,
390             }
391
392         groups: List[List[Tuple[int, float]]] = []
393         group = [active[0]]
394         for af in active[1:]:
395             if af[0] - group[-1][0] <= max_gap_frames:
396                 group.append(af)
397             else:
398                 groups.append(group)
399                 group = [af]
400         groups.append(group)
401
402         if max_window_duration > 0:
403             groups = TrackNetRunner._split_overlong_groups(

```

```

404         groups,
405         int(max_window_duration * fps),
406         int(min_split_gap * fps),
407         hard_cap=window_hard_cap,
408     )
409
410     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
411     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
412     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
413     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
414     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
415     if window_inactivity_end > 0:
416         tw = int(window_inactivity_end * fps)
417         groups = [
418             TrackNetRunner._trim_inactive_tail(
419                 g,
420                 tw,
421                 inactivity_min_density,
422                 inactivity_rally_thresh,
423                 temporal_density=inactivity_temporal_density,
424             )
425             for g in groups
426         ]
427
428     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
429     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
430     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
431     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
432     forced_flags = [False] * len(groups)
433     if window_blob_fallback and max_window_duration > 0:
434         groups, forced_flags = TrackNetRunner._blob_fallback_chop(
435             groups, int(max_window_duration * fps), window_blob_factor
436         )
437
438     windows = []
439     for g, forced in zip(groups, forced_flags):
440         w = _finalize(g)
441         if forced:
442             w["forced_cut"] = True
443             windows.append(w)
444     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
445
446     @staticmethod
447     def _trim_inactive_tail(
448         group: List[Tuple[int, float]],
449         trail_frames: int,
450         min_density: float,
451         rally_thresh: float,
452         temporal_density: bool = False,
453     ) -> List[Tuple[int, float]]:
454         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
455         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
456         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``

```

```

457         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
458         ``trail_frames`` window holds  $\geq$  ``min_density`` fraction of fast frames; trim
back to the
459         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
460         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
461         exists at all the group is kept whole (fail-safe, no over-trim). 0(n) two-
pointer; the START
462         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding).
463
464         ``temporal_density`` (default False = legacy/promoted behaviour) selects the
density
465         DENOMINATOR (code-review finding B). ``group`` carries only active frames, so
the legacy
466         denominator ? the COUNT of active samples in the trailing window (``i - lo +
1``) ? reads a
467         lone fast sample after a long tracking gap as 100% dense and holds the tail
open. When True,
468         the denominator is the trailing window's TEMPORAL length instead, so sparse
tracking can't
469         inflate density. With DENSE tracking the two are identical, so this is a no-op
on
470         well-tracked clips ? hence it ships flag-gated default-OFF until measured on the
golden set
471         at the SERVED operating point (R4 is currently default-ON in prod; see
RALLY_DETECTION_FIXES_PLAN)."""
472         if trail_frames <= 0 or len(group) < 2:
473             return group
474         frames = [f for f, _ in group]
475         fast = [1 if v > rally_thresh else 0 for _, v in group]
476         lo = 0
477         fast_sum = 0
478         last_dense = -1
479         for i in range(len(frames)):
480             fast_sum += fast[i]
481             while frames[lo] <= frames[i] - trail_frames:
482                 fast_sum -= fast[lo]
483                 lo += 1
484             if temporal_density:
485                 # Trailing window's TEMPORAL length: full ``trail_frames`` once that
much footage
486                 # has elapsed, else the footage seen so far. Sparse tracking can't
inflate it.
487                 denom = min(trail_frames, frames[i] - frames[0] + 1)
488             else:
489                 denom = (
490                     i - lo + 1
491                 ) # legacy: count of active samples in the trailing window
492             if denom > 0 and fast_sum / denom  $\geq$  min_density:
493                 last_dense = i
494         return group[: last_dense + 1] if last_dense  $\geq$  0 else group
495
496     @staticmethod
497     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
498         """Index of the active frame nearest the group's temporal midpoint (used to chop
a
499         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
500         mid = (group[0][0] + group[-1][0]) / 2.0
501         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
502         return split_i
503
504     @staticmethod

```

```

505     def _split_overlong_groups(
506         groups: List[List[Tuple[int, float]]],
507         max_win_frames: int,
508         min_split_frames: int,
509         hard_cap: bool = False,
510     ) -> List[List[Tuple[int, float]]]:
511         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
512         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
513         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
514         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
515         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
516         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
517         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
518         if max_win_frames <= 0:
519             return groups
520         out: List[List[Tuple[int, float]]] = []
521         work = list(groups)
522         while work:
523             g = work.pop()
524             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
525                 out.append(g)
526                 continue
527             best_i, best_gap = -1, -1
528             for i in range(1, len(g)):
529                 gap = g[i][0] - g[i - 1][0]
530                 if gap > best_gap:
531                     best_gap, best_i = gap, i
532             if best_i < 0 or best_gap < min_split_frames:
533                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
534                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
535                 if hard_cap:
536                     split_i = TrackNetRunner._midpoint_split_index(g)
537                     work.append(g[:split_i])
538                     work.append(g[split_i:])
539                 else:
540                     out.append(g)
541                     continue
542                 work.append(g[:best_i])
543                 work.append(g[best_i:])
544             out.sort(key=lambda grp: grp[0][0])
545         return out
546
547     @staticmethod
548     def _blob_fallback_chop(
549         groups: List[List[Tuple[int, float]]],
550         max_win_frames: int,
551         blob_factor: float = 4.0,
552     ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
553         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
554         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
555         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
556         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?

```

```

557         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
558         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
559         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
560         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
561         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
562         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
563         that piece came from a forced chop. Iterative worklist to bound stack depth."""
564         if max_win_frames <= 0:
565             return groups, [False] * len(groups)
566         blob_frames = max_win_frames * max(1.0, blob_factor)
567         out: List[List[Tuple[int, float]]] = []
568         flags: List[bool] = []
569         forced = 0
570         for g in groups:
571             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
572                 out.append(g)
573                 flags.append(False)
574                 continue
575             pieces: List[List[Tuple[int, float]]] = []
576             work = [g]
577             while work:
578                 h = work.pop()
579                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
580                     pieces.append(h)
581                     continue
582                 split_i = TrackNetRunner._midpoint_split_index(h)
583                 work.append(h[:split_i])
584                 work.append(h[split_i:])
585             pieces.sort(key=lambda grp: grp[0][0])
586             out.extend(pieces)
587             flags.extend([True] * len(pieces))
588             forced += len(pieces) - 1
589         if forced:
590             logger.warning(
591                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
592                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
593                 "motion), not a bigger cap",
594                 forced,
595             )
596         return out, flags
597

```

16. assistant (2026-07-07T15:10:27.884Z)

Now the config models, the capabilities probe in main.py, and the segmenter/fusion files.

17. user (2026-07-07T15:10:29.887Z)

```

340
341     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
342     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
343     API_VERSION = 1
344
345
346     def _engine_version() -> str:
347         """Installed package version (wheel/editable); 'unknown' if not importable as a

```

```

distribution."""
348         try:
349             return _pkg_version("badminton-highlight-indexer")
350         except PackageNotFoundError:
351             return "unknown"
352
353
354     @app.get("/api/version")
355     def api_version():
356         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
357         return {
358             "engine_version": _engine_version(),
359             "api_version": API_VERSION,
360             "ui": _bundled_ui_version(),
361         }
362
363
364     @app.get("/api/capabilities")
365     def capabilities(request: Request):
366         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
367         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
368         value. Best-effort + never raises (the wizard degrades gracefully)."""
369         cfg = request.app.state.config
370         indexing = getattr(cfg, "indexing", None)
371         deployment = getattr(cfg, "deployment", None)
372         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
373         ffprobe = shutil.which(ffprobe_bin())
374         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
375         wasb = getattr(indexing, "wasb", None)
376         weights_path = (
377             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
378         )
379         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
380         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
381         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
382         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
383         nvidia_smi = shutil.which("nvidia-smi")
384         return {
385             "api_version": API_VERSION,
386             "engine_version": _engine_version(),
387             "segmenters": sorted(segmenter_registry.list_available().keys()),
388             "default_segmenter": getattr(indexing, "default_segmenter", None),
389             "ffmpeg": {
390                 "ffmpeg": bool(ffmpeg),
391                 "ffprobe": bool(ffprobe),
392                 "ffmpeg_path": ffmpeg,
393                 "ffprobe_path": ffprobe,
394             },
395             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
396             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
397             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
398             **uploads_api.upload_caps(cfg), # #480 item 5: the SPA pre-flight reads
max_upload_mb
399             "deployment": {
400                 "profile": getattr(deployment, "profile", "") or "",
401                 "compute_target": getattr(deployment, "compute_target", "") or "",
402                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows

```

```

403         # the "run in the cloud" toggle only when true (else it's local-only).
404         "cloud_available": _cloud_available(cfg),
405     },
406     # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
407     # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
408     "storage": storage_capabilities(
409         getattr(cfg, "storage", None), local_free_path=output_base(cfg)
410     ),
411 }
412
413
414 def _friendly_video_name(filepath: str) -> str:
415     """A human name for a video from its stored path/URI ? the filename stem
416     (``?/match_2026.mp4`` ?
417     ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
418     (rsplit on ``/``).
419     Falls back to the whole ``filepath`` if there's no basename."""
420     base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
421     stem = os.path.splitext(base)[0]
422     return stem or base or str(filepath or "")
423
424 def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
425     """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
426     None)``. The
427     ``resolution_fps_check`` validator already probed ffmpeg and stored
428     ``details={width,height,fps}``
429     on its result (every branch) ? so the videos row carries it with no schema change /
430     re-probe."""
431     for r in validation_results or []:
432         if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
433             d = r.get("details") or {}
434             w, h, fps = d.get("width"), d.get("height"), d.get("fps")
435             resolution = f"{w}x{h}" if w and h else None
436             return (fps if isinstance(fps, (int, float)) else None), resolution
437     return None, None
438
439 def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
440     """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
441     ``id``/``filepath`` (an
442     existing test + any old client rely on them) and adds the fields the SPA library +
443     the persistent
444     "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
445     friendly
446     ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
447     ``local_path``,
448     ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
449     ffmpeg
450     validation details) so the library cards can show them. Mirrors the mock engine's
451     shape so mock and
452     real agree."""
453     fp = row.get("filepath") or ""
454     fps, resolution = _video_dims(row.get("validation_results"))
455     return {
456         **row,
457         "video_id": row.get("id"),
458         "match_name": _friendly_video_name(fp),

```

18. user (2026-07-07T15:10:30.289Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2

```



```

3 Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4 TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5 do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6 shuttle-seeded window via two overridable seams the engine exposes:
7
8 - ``enrich_candidate_stats`` ? always adds the net-crossing count (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13 torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14 model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15 via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16 - ``select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19 Persisted candidates remain the full superset regardless, so the offline
experiment
20 harness can re-score any variant.
21
22 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26 experiment harness (EXPERIMENT_HARNESS.md), never silently.
27 """
28
29 from typing import Any, Dict, List, Optional, Tuple
30
31 from backend.config.models import FusionConfig, IndexingConfig
32 from backend.pipeline.detectors import fusion_features as ff
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors.base import DetectorRunner
35 from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
36 from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
37 from backend.pipeline.segmenters.base import segmenter_registry
38 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
39
40
41 class FusionSegmenter(TrajectoryHybridSegmenter):
42     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
43
44     family = "fusion"
45     display_label = "Fusion"
46
47     def __init__(self, db: Any, config: Any):
48         super().__init__(db, config)
49         # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
50         self._person: Optional[Any] = None
51         # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
52         # over the whole trajectory once per candidate window (it depends only on
`points`).
53         self._axis_cache_key: Optional[int] = None

```

```

54         self._axis_cache: Optional[tuple] = None
55
56     @property
57     def name(self) -> str:
58         return "fusion_hybrid"
59
60     @property
61     def description(self) -> str:
62         return (
63             "Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
64             "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and
the "
65             "AI provider sets semantic boundaries."
66         )
67
68     def _build_runner(
69         self, idx_cfg: IndexingConfig
70     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
71         if isinstance(idx_cfg, dict):
72             idx_cfg = IndexingConfig(**idx_cfg)
73         substrate = idx_cfg.fusion.substrate
74         if substrate == "tracknet":
75             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
76             return TrackNetRunner(cfg), {
77                 "weights_path": cfg.weights_path,
78                 "queue_length": cfg.queue_length,
79                 "fusion_substrate": "tracknet",
80             }
81         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
82         return WasbRunner(wcfg), {
83             "weights_path": wcfg.weights_path,
84             "fusion_substrate": "wasb",
85         }
86
87     def _build_native_runner(
88         self, idx_cfg: IndexingConfig
89     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
90         # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
91         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
92         substrate = idx_cfg.fusion.substrate
93         if substrate != "wasb":
94             raise NotImplementedError(
95                 "detector_impl='native' supports the WASB substrate only ? set "
96                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet)."
97             )
98         from backend.pipeline.detectors.native_wasb_runner import (
99             NativeWasbConfig,
100             NativeWasbRunner,
101         )
102
103         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
104         return NativeWasbRunner(cfg), {
105             "weights_path": cfg.weights_path,
106             "detector_impl": "native",
107             "fusion_substrate": "wasb",
108             "device": cfg.device,
109             # Audit the decode path alongside the CPU/nvdec-isolated cache (#506).
110             "decode_backend": cfg.decode_backend,
111         }
112
113     def _enrich_candidate_stats(
114         self,

```

```

115         cw: Dict[str, Any],
116         points: List[Any],
117         fps: float,
118         frame_width: float,
119         video_path: str,
120         idx_cfg: IndexingConfig,
121     ) -> Dict[str, Any]:
122         stats = super()._enrich_candidate_stats(
123             cw, points, fps, frame_width, video_path, idx_cfg
124         )
125         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
126         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
127         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
128         gated = rally_gate.gate_windows(
129             points,
130             [(cw["start"], cw["end"])],
131             fps,
132             min_crossings=0,
133             estimate=self._axis_estimate(points),
134         )
135         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
136
137         fcfg = idx_cfg.fusion
138         if fcfg.cue_flags.get("person", False):
139             stats.update(
140                 self._person_features(cw, points, fps, frame_width, video_path, fcfg)
141             )
142         return stats
143
144         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
145         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
146         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
147         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
148         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
149         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
150         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
151
152         def _axis_estimate(self, points: List[Any]) -> tuple:
153             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
154             computed once per video, not once per candidate window (it depends only on
points)."""
155             key = id(points)
156             if self._axis_cache_key != key or self._axis_cache is None:
157                 self._axis_cache_key = key
158                 self._axis_cache = rally_gate.estimate_play_axis(points)
159             est = self._axis_cache
160             assert est is not None
161             return est
162
163         def _person_features(
164             self,
165             cw: Dict[str, Any],
166             points: List[Any],
167             fps: float,
168             frame_width: float,

```

```

169         video_path: str,
170         fcfg: "FusionConfig",
171     ) -> Dict[str, float]:
172         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
173         frame range and compute the scale/translation-invariant person features. Lazily
builds
174         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
175         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
176         from backend.pipeline.detectors.person_detector import PersonDetector
177
178         if self._person is None:
179             self._person = PersonDetector(self.config)
180
181         frame_skip = self.config.indexing.yolo_frame_skip
182         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
183         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
184         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
185         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
186         fw = det["frame_width"] or frame_width
187         fh = det["frame_height"]
188         # If we can't trust the frame dims, every normalised feature would silently
collapse
189         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
190         if fw <= 0 or fh <= 0:
191             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
192
193         poly = fcfg.court_polygon
194         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
195         net = (fcfg.net_axis, float(fcfg.net_position))
196         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
197         return ff.compute_person_features(
198             det["frames"], shuttle_in, fw, fh, court_polygon=polygon, net=net
199         )
200
201     def _select_for_handoff(
202         self,
203         windows: List[Dict[str, Any]],
204         window_stats: List[Dict[str, Any]],
205         idx_cfg: IndexingConfig,
206     ) -> List[int]:
207         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
208         (0) keep every window ? identical to the substrate. Persisted candidates are
209         untouched (full superset).
210
211         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
212         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
213         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
214         set rather than emit a video with zero rallies. This makes Gate A/B strictly
215         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
216         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
217         below it always wins on an otherwise-empty video. Observed on real footage
218         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
219         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""

```

```

220         fcfg = idx_cfg.fusion
221         min_crossings = int(fcfg.min_crossings)
222         min_players = int(fcfg.min_players)
223         person_on = bool(fcfg.cue_flags.get("person", False))
224
225         keep: List[int] = []
226         for i, st in enumerate(window_stats):
227             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
228                 continue
229             if (
230                 person_on
231                 and min_players > 0
232                 and st.get("players_in_court", 0.0) < min_players
233             ):
234                 continue
235             keep.append(i)
236
237         if not keep and window_stats:
238             # Gating emptied a non-empty video ? fall back to the ungated set.
239             return list(range(len(window_stats)))
240         return keep
241
242
243     segmenter_registry.register(FusionSegmenter)
244

```

19. assistant (2026-07-07T15:11:41.275Z)

Let me read the trajectory hybrid (runner resolution), config models (defaults + substrate type), and the call site, plus the tracknet/wasb hybrids.

20. user (2026-07-07T15:11:42.781Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20
21    import concurrent.futures
22    import logging
23    import os
24    import time
25    from typing import Any, Dict, List, Optional, Tuple, Union
26
27    import cv2
28
29    from backend.config.models import IndexingConfig
30    from backend.pipeline.detectors.base import DetectorRunner
31    from backend.pipeline.segmenters.base import BasePlaySegmenter
32    from backend.providers import provider_registry

```

```

33
34 # Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
35 from backend.results import Failure
36 from backend.sports import sport_registry
37 from backend.utils.paths import output_base
38 from backend.utils.video import extract_video_chunk, get_video_duration
39
40 logger = logging.getLogger(__name__)
41
42
43 def _fmt_ts(seconds: float) -> str:
44     seconds = max(0, int(seconds))
45     h, rem = divmod(seconds, 3600)
46     m, s = divmod(rem, 60)
47     return f"{h:02d}:{m:02d}:{s:02d}"
48
49
50 class TrajectoryHybridSegmenter(BasePlaySegmenter):
51     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
52 boundaries."""
53
54     #: config section under `indexing` (velocity_threshold lives there), the
55     #: output subdir for trajectories, and the metadata detector-tag prefix.
56     family: str = ""
57     #: human-readable label used in log lines.
58     display_label: str = ""
59
60     def _build_runner(
61         self, idx_cfg: IndexingConfig
62     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
63         """Return (runner, detector-specific detection_params extras) for the default
64 (WSL) path."""
65         raise NotImplementedError
66
67     def _build_native_runner(
68         self, idx_cfg: IndexingConfig
69     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
70         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
71 with a
72 native runner override this; the base refuses with a clear message (M3.5: WASB
73 only)."""
74         raise NotImplementedError(
75             f"detector_impl='native' is not supported for the "
76             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
77 has a "
78             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
79 )
80
81     def _resolve_runner(
82         self, idx_cfg: IndexingConfig
83     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
84         """Resolve the detector runner, honouring the CPU-stub and Linux-native
85 overrides.
86
87         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
88         ``indexing.detector_impl``:
89         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
90 pipeline
91 swaps
92         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
93 swaps
94         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
95         ``_build_native_runner``. Anything else (``"auto"``) delegates to
96         ``_build_runner``
97         (the WSL runner) so the default production path is unchanged
98         (``docs/DECOUPLED_COMPUTE/``).

```

```

88         """
89         # Safety net: accept raw dicts from legacy / test callers.
90         if isinstance(idx_cfg, dict):
91             idx_cfg = IndexingConfig(**idx_cfg)
92         impl = (
93             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
94         ).lower()
95         if impl == "stub":
96             from backend.pipeline.detectors.stub_runner import (
97                 StubConfig,
98                 StubDetectorRunner,
99             )
100
101             logger.info(
102                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
103                 self.display_label or "trajectory",
104             )
105             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg), {
106                 "detector_impl": "stub"
107             })
108         if impl in ("native", "native_wasb", "wasb_native"):
109             logger.info(
110                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
111                 self.display_label or "trajectory",
112             )
113             return self._build_native_runner(idx_cfg)
114         return self._build_runner(idx_cfg)
115
116     @staticmethod
117     def _probe_fps_width(video_path: str) -> tuple:
118         """Return (fps, frame_width) for a video, with sensible fallbacks."""
119         cap = cv2.VideoCapture(video_path)
120         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
121         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
122         cap.release()
123         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
124
125     @staticmethod
126     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
127         """Provider-reported exchange count, else a duration-based estimate.
128
129         One canonical implementation ? the twins had drifted (wasb relied on
130         ``int(None)`` raising into the fallback; same result, by accident).
131         """
132         shot_count = segment.get("shuttle_exchange_count")
133         if shot_count is None:
134             return max(3, int(duration / 0.8))
135         try:
136             return int(shot_count)
137         except (ValueError, TypeError):
138             return max(3, int(duration / 0.8))
139
140     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
141     -----
142     def _enrich_candidate_stats(
143         self,
144         cw: Dict[str, Any],
145         points: List[Any],
146         fps: float,
147         frame_width: float,
148         video_path: str,
149         idx_cfg: "IndexingConfig",
150     ) -> Dict[str, Any]:
151         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
152         BASE

```

```

151         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
152         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
153         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
154         return {
155             "peak_velocity": cw["peak_velocity"],
156             "mean_velocity": cw["mean_velocity"],
157             "active_frame_count": cw["active_frame_count"],
158             "track_id_count": cw["track_id_count"],
159         }
160
161     def _select_for_handoff(
162         self,
163         windows: List[Dict[str, Any]],
164         window_stats: List[Dict[str, Any]],
165         idx_cfg: "IndexingConfig",
166     ) -> List[int]:
167         """Indices of the candidate windows that proceed to the AI handoff (and thus may
168         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
169         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
170         Persisted candidates are NOT affected ? they remain the full superset for
offline
171         re-scoring."""
172         return list(range(len(windows)))
173
174     def process_video( # type: ignore[override]
175         self,
176         video_id: str,
177         video_path: str,
178         player_pool: Optional[List[str]] = None,
179         max_frames: Optional[int] = None,
180     ) -> "Union[List[Dict[str, Any]], Failure]":
181         # Result contract (the ABC declares Success|Failure). Anything that means the
run could
182         # NOT actually complete ? an unrunnable config (unknown sport/provider, missing
API key),
183         # an unreadable video, an unavailable detector (native-not-supported), the env
healthcheck
184         # not ready, or run_predict timing out / aborting with no trajectory ? returns a
``Failure``
185         # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
186         # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
187         # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
188         label = self.display_label
189         # --- Sport profile + AI provider wiring (identical across segmenters) ---
190         sport_name = self.config.sport
191         try:
192             sport_profile = sport_registry.get(sport_name)
193         except KeyError as e:
194             logger.error(str(e))
195             # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
196             return Failure(str(e))
197
198         idx_cfg = self.config.indexing
199         skip_ai = bool(idx_cfg.skip_ai_handoff)
200         provider_name = self.config.ai_provider
201         if skip_ai:
202             model_name = "local-noai"
203             logger.info(
204                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
205                 "be emitted as rallies; no %s handoff, no API key needed.",

```



```

206         label,
207         provider_name,
208     )
209 else:
210     model_name = self.config.ai_model
211     api_key_env_var = f"{provider_name.upper()}_API_KEY"
212     api_key = os.environ.get(api_key_env_var)
213     if not api_key:
214         from backend.pipeline.segmenters._keyhint import api_key_hint
215
216         msg = (
217             f"{api_key_env_var} environment variable is not set! "
218             f"To run the {provider_name} handoff, set your API key. "
219             + api_key_hint(api_key_env_var)
220         )
221         logger.error(msg)
222         # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
223         # marked failed instead of a silent empty "success" (mirrors gemini; the
worker's
224         # own 0-segment diagnostic names this missing-key case as a top cause).
225         return Failure(msg, is_recoverable=True)
226     try:
227         provider = provider_registry.get(
228             provider_name, api_key=api_key, model_name=model_name
229         )
230     except KeyError as e:
231         logger.error(str(e))
232         return Failure(str(e))
233
234     video_duration = get_video_duration(video_path)
235     if video_duration <= 0:
236         logger.error("Invalid video duration.")
237         # An unreadable/broken video, not a 0-rally match.
238         return Failure(
239             message=f"{label}: invalid/unreadable video duration
({video_duration}).",
240         )
241     fps, frame_width = self._probe_fps_width(video_path)
242
243     # --- Runner config + windowing thresholds ---
244     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
245     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
246     # it delegates to the subclass _build_runner (the real WSL/native runner).
247     try:
248         runner, runner_params = self._resolve_runner(idx_cfg)
249     except NotImplementedError as e:
250         # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
251         # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
252         # 0-rally match ? refuse cleanly as a Failure instead of crashing or a false
success.
253         logger.error("%s: %s", label, e)
254         return Failure(message=f"{label}: {e}")
255
256     velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
257     merge_gap = idx_cfg.dead_time_merge_gap
258     min_window_duration = idx_cfg.min_action_window_duration
259     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
260     max_window_duration = idx_cfg.max_window_duration
261     window_min_split_gap = idx_cfg.window_min_split_gap
262     window_hard_cap = idx_cfg.window_hard_cap
263     window_inactivity_end = idx_cfg.window_inactivity_end

```

```

264     inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
265     inactivity_min_density = idx_cfg.inactivity_min_density
266     inactivity_temporal_density = idx_cfg.inactivity_temporal_density
267     window_blob_fallback = idx_cfg.window_blob_fallback
268     window_blob_factor = idx_cfg.window_blob_factor
269     handoff_padding = idx_cfg.ai_handoff_padding
270     ai_max_workers = idx_cfg.ai_max_workers
271     log_candidates = idx_cfg.log_yolo_candidates
272     store_candidates = idx_cfg.store_yolo_candidates
273
274     detection_params = {
275         "detector": self.name,
276         "velocity_thresh": velocity_thresh,
277         "merge_gap": merge_gap,
278         "min_action_window_duration": min_window_duration,
279         **runner_params,
280     }
281
282     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
283     ok, msg = runner.healthcheck()
284     if not ok:
285         logger.error("%s detector environment not ready: %s", label, msg)
286         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
287         # failed/retryable instead of writing an empty-but-"successful" result.
288         return Failure(
289             message=f"{label} detector environment not ready: {msg}",
290             is_recoverable=True,
291         )
292     logger.info("%s detector env OK: %s", label, msg)
293
294     # --- Phase 2: trajectory -> candidate rally windows ---
295     # Trajectory detectors process the whole video in one pass (they carry
296     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
297     t_start = time.time()
298     out_dir = os.path.join(output_base(self.config), self.family)
299     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300     if not csv_path:
301         logger.error("%s produced no trajectory; aborting.", label)
302         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
303         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
304         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305         # rather than a silent 0-segment "success" (the wf294 incident).
306         return Failure(
307             message=(
308                 f"{label} detector produced no trajectory (timed out or aborted) ? "
309                 "detector failure, not a 0-rally video."
310             ),
311             is_recoverable=True,
312         )
313
314     points = runner.parse_trajectory_csv(csv_path)
315     logger.info(
316         "Parsed %d trajectory points (%d visible).",
317         len(points),
318         sum(1 for p in points if p.visible),
319     )
320
321     all_candidate_windows = runner.trajectory_to_action_windows(
322         points,
323         fps=fps,
324         frame_width=frame_width,

```

```

325         chunk_start=0.0,
326         velocity_thresh=velocity_thresh,
327         merge_gap=merge_gap,
328         min_window_duration=min_window_duration,
329         chunk_index=0,
330         max_window_duration=max_window_duration,
331         min_split_gap=window_min_split_gap,
332         window_hard_cap=window_hard_cap,
333         window_inactivity_end=window_inactivity_end,
334         inactivity_rally_thresh=inactivity_rally_thresh,
335         inactivity_min_density=inactivity_min_density,
336         inactivity_temporal_density=inactivity_temporal_density,
337         window_blob_fallback=window_blob_fallback,
338         window_blob_factor=window_blob_factor,
339     )
340     logger.info(
341         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
342         label,
343         time.time() - t_start,
344         len(all_candidate_windows),
345     )
346
347     if log_candidates:
348         for cw in all_candidate_windows:
349             logger.info(
350                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
351                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
352                 f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
353             )
354
355     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
356     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
357     # persistence and the handoff gate below.
358     window_stats = [
359         self._enrich_candidate_stats(
360             cw, points, fps, frame_width, video_path, idx_cfg
361         )
362         for cw in all_candidate_windows
363     ]
364
365     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
366     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
367     if store_candidates:
368         for i, cw in enumerate(all_candidate_windows):
369             candidate_ids[i] = self.db.add_candidate_segment(
370                 video_id,
371                 detector=self.name,
372                 start_time=cw["start"],
373                 end_time=cw["end"],
374                 chunk_index=cw["chunk_index"],
375                 confidence=min(1.0, cw["peak_velocity"]),
376                 stats=window_stats[i],
377                 detection_params=detection_params,
378             )
379
380     if not all_candidate_windows:
381         return []
382
383     # --- Phase 3: AI provider handoff over padded candidate clips ---
384     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
385     # candidates above stay the full superset ? only the served play_segments are

```

```

gated.
386         handoff_indices = self._select_for_handoff(
387             all_candidate_windows, window_stats, idx_cfg
388         )
389
390     ai_segments: List[dict] = []
391     if skip_ai:
392         # Fully-local: the selected candidate windows ARE the rallies ? emit them
393         # (no clip extraction, no upload). Boundaries are the local detector's; shot
394         # falls back to the duration-based estimate (no AI exchange count).
395         ai_segments = [
396             {
397                 "start_time": all_candidate_windows[wi]["start"],
398                 "end_time": all_candidate_windows[wi]["end"],
399                 "shuttle_exchange_count": None,
400                 "shot_count_confidence": "LocalNoAI",
401                 "_candidate_id": candidate_ids[wi],
402             }
403             for wi in handoff_indices
404         ]
405         logger.info(
406             "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
407             "
408             "(no AI handoff).",
409             len(ai_segments),
410         )
411     else:
412         handoff_dir = os.path.join(
413             output_base(self.config), f"chunks_{provider_name}_handoff"
414         )
415         os.makedirs(handoff_dir, exist_ok=True)
416         logger.info(
417             "Phase 3: %s validation on %d candidate windows...",
418             provider_name,
419             len(handoff_indices),
420         )
421         def process_candidate_window(
422             wi: int, window: Dict[str, Any]
423         ) -> Tuple[List[dict], bool]:
424             """Returns (valid_segments, errored) ? errored=True when this window
425             yielded NO
426             verdict (clip extraction failed, or the provider returned
427             metrics['error'])."""
428             w_start = max(0, window["start"] - handoff_padding)
429             w_end = min(video_duration, window["end"] + handoff_padding)
430             w_dur = w_end - w_start
431             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
432             if not extract_video_chunk(
433                 video_path, w_start, w_dur, w_path, reencode=True
434             ):
435                 return [], True # extraction failed ? no verdict for this window
436             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
437             segments, metrics = provider.analyze_video(
438                 w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
439             )
440             # B2: a provider failure returns [] with metrics["error"] set ? that is
441             # "we don't know", NOT "no rallies in this window". Surface it loudly so
442             a
443             # partial run isn't mistaken for a complete one (mirrors the gemini
444             segmenter).
445             errored = bool(metrics.get("error"))
446             if errored:

```

```

443         logger.warning(
444             "[Handoff %d] provider error ? window %d skipped: %s",
445             wi + 1,
446             wi,
447             metrics["error"],
448         )
449     valid = []
450     for seg in segments:
451         try:
452             seg["start_time"] = float(seg["start_time"]) + w_start
453             seg["end_time"] = float(seg["end_time"]) + w_start
454             seg["_candidate_id"] = candidate_ids[wi]
455             valid.append(seg)
456         except (ValueError, TypeError, KeyError) as e:
457             logger.warning(
458                 "Dropping segment with unparseable timestamps: %s ? %s",
459                 seg,
460                 e,
461             )
462     try:
463         os.remove(w_path)
464     except OSError:
465         pass
466     return valid, errored
467
468 with concurrent.futures.ThreadPoolExecutor(
469     max_workers=ai_max_workers
470 ) as executor:
471     futures = [
472         executor.submit(
473             process_candidate_window, wi, all_candidate_windows[wi]
474         )
475         for wi in handoff_indices
476     ]
477     window_results = [future.result() for future in futures]
478     for valid, _errored in window_results:
479         ai_segments.extend(valid)
480     handoff_failed = sum(1 for _valid, errored in window_results if errored)
481     try:
482         os.rmdir(handoff_dir)
483     except OSError:
484         pass
485     logger.info(
486         "Phase 3 completed. AI confirmed %d semantic play segments.",
487         len(ai_segments),
488     )
489     # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
490     # (provider error / clip extraction failed), the run has zero information ?
491     # not a 0-rally match. A single window that RAN and returned no rallies is a
legit empty
492     # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
493     if handoff_indices and handoff_failed == len(handoff_indices):
494         msg = (
495             f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
496             "window(s) errored (provider error / clip extraction failed) ?
incomplete "
497             "run, not a 0-rally match."
498         )
499         logger.error(msg)
500         return Failure(message=msg, is_recoverable=True)
501

```

```

502         # --- Phase 4: DB population + candidate linking ---
503         segments_data = []
504         for segment in ai_segments:
505             try:
506                 start = float(segment["start_time"])
507                 end = float(segment["end_time"])
508                 dur = end - start
509                 metadata = {
510                     "shot_count": self._shot_count_from(segment, dur),
511                     "shot_count_confidence": segment.get(
512                         "shot_count_confidence", "Unknown"
513                     ),
514                     "detector": f"{self.family}-hybrid-{model_name}",
515                 }
516                 segment_id = self.db.add_play_segment(
517                     video_id, start, end, None, None, metadata
518                 )
519                 if segment_id:
520                     candidate_id = segment.get("_candidate_id")
521                     if candidate_id is not None:
522                         self.db.link_candidate_to_segment(candidate_id, segment_id)
523                     segments_data.append(
524                         {
525                             "id": segment_id,
526                             "start_time": start,
527                             "end_time": end,
528                             "duration": dur,
529                             "metadata": metadata,
530                         }
531                     )
532             except Exception as e:
533                 logger.warning("Skipping invalid segment struct: %s", e)
534
535         return segments_data
536

```

21. user (2026-07-07T15:11:43.284Z)

```

1         """WASB-based hybrid play segmenter.
2
3         Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4         WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5         badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7         For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8         candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9         min_window_duration) are what get tuned against golden rally labels.
10
11         The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12         lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13         WASB runner and identity.
14         """
15
16         from typing import Any, Dict, Tuple
17
18         from backend.config.models import IndexingConfig
19         from backend.pipeline.detectors.base import DetectorRunner
20         from backend.pipeline.detectors.wasb_runner import WasbConfig, WasbRunner
21         from backend.pipeline.segmenters.base import segmenter_registry
22         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
23
24
25         class WasbHybridSegmenter(TrajectoryHybridSegmenter):
26             family = "wasb"
27             display_label = "WASB"

```

```

28
29     @property
30     def name(self) -> str:
31         return "wasb_hybrid"
32
33     @property
34     def description(self) -> str:
35         return (
36             "Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
37             "weights) for precise candidate rallies + AI provider for semantic
boundaries."
38         )
39
40     def _build_runner(
41         self, idx_cfg: IndexingConfig
42     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
43         cfg = WasbConfig.from_indexing_cfg(idx_cfg)
44         return WasbRunner(cfg), {"weights_path": cfg.weights_path}
45
46     def _build_native_runner(
47         self, idx_cfg: IndexingConfig
48     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
49         # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
50         # seam (imported only when detector_impl='native' is actually selected).
51         from backend.pipeline.detectors.native_wasb_runner import (
52             NativeWasbConfig,
53             NativeWasbRunner,
54         )
55
56         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
57         return NativeWasbRunner(cfg), {
58             "weights_path": cfg.weights_path,
59             "detector_impl": "native",
60             "device": cfg.device,
61             # Record the decode path so persisted candidates are auditable: the cpu and
62             # nvdec_resident caches are non-interchangeable (#506), so downstream evals
/
63             # dataset exports must be able to tell which decode produced a candidate.
64             "decode_backend": cfg.decode_backend,
65         }
66
67
68     segmenter_registry.register(WasbHybridSegmenter)
69

```

22. user (2026-07-07T15:11:43.838Z)

```

1     """TrackNet-based hybrid play segmenter.
2
3     Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4     in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5     tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6     purpose-built trajectory model localises it far more precisely, giving cleaner
7     candidate rallies than person-velocity heuristics.
8
9     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10    lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11    TrackNet runner and identity.
12    """
13
14    from typing import Any, Dict, Tuple
15
16    from backend.config.models import IndexingConfig

```

```

17 from backend.pipeline.detectors.base import DetectorRunner
18 from backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
19 from backend.pipeline.segmenters.base import segmenter_registry
20 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
21
22
23 class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
24     family = "tracknet"
25     display_label = "TrackNet"
26
27     @property
28     def name(self) -> str:
29         return "tracknet_hybrid"
30
31     @property
32     def description(self) -> str:
33         return (
34             "Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
35             "candidate rallies + AI provider for high-precision semantic boundaries."
36         )
37
38     def _build_runner(
39         self, idx_cfg: IndexingConfig
40     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
41         cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
42         return TrackNetRunner(cfg), {
43             "weights_path": cfg.weights_path,
44             "queue_length": cfg.queue_length,
45         }
46
47
48 segmenter_registry.register(TrackNetHybridSegmenter)
49

```

23. assistant (2026-07-07T15:12:54.112Z)

Let me read the config models, the `\_maybe\_dispatch\_remote` call site, and the SPA type/client files to close the fidelity questions.

24. user (2026-07-07T15:12:58.040Z)

```

=== FusionConfig / substrate ===
69:  """Typed config for the live WASB shuttle substrate (indexing.wasb).
179:  # Which trajectory substrate seeds the windows (shuttle stays primary).
180:  substrate: Literal["wasb", "tracknet"] = "wasb"
182:  # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
=== wasb weights_path / tracknet weights_path / detector_impl defaults ===
51:  weights_path: str = ""
80:  weights_path: str = (
87:  # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
168:class FusionConfig(BaseModel):
202:class IndexingConfig(BaseModel):
210:    detector_impl: str = "auto"
463:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.

```

25. user (2026-07-07T15:12:59.710Z)

```

=== _maybe_dispatch_remote call site ===
backend/main.py:231:# backend.main ? `import backend.main as m; m._execute_processing(...)` , the
job worker's
backend/main.py:238:     _execute_processing,
backend/main.py:241:     _maybe_dispatch_remote,
backend/main.py:268:     # default (deployment.compute_target). Only honoured for these two

```



```

values; see _maybe_dispatch_remote.
backend/orchestration.py:3:The full hash -> validate -> segment -> report pipeline
(``_execute_processing``), the
backend/orchestration.py:4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
backend/orchestration.py:13:...))`` and ``import backend.main as m; m._execute_processing(...)``
keep working unchanged.
backend/orchestration.py:652:def _execute_processing(
backend/orchestration.py:660:     test code that calls ``_execute_processing(req)`` still works.
backend/orchestration.py:667:     # Backward-compatible shim: if called as
_execute_processing(req) (old pattern),
backend/orchestration.py:670:         # Old signature: _execute_processing(req, progress=None)
backend/orchestration.py:721:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend/orchestration.py:798:         remote = _maybe_dispatch_remote(

```

26. user (2026-07-07T15:13:10.718Z)

```

40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0
66
67
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72         these knobs actually take effect ? previously this whole block was silently dropped
73         because nested config sections defaulted to extra='ignore'.
74         """
75
76         wsl_distro: str = "Ubuntu"
77         repo_dir: str = "~/models/WASB-SBDT"
78         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79         conda_env: str = "wasb"
80         weights_path: str = (
81             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82         )
83         sport: str = "badminton"
84         wsl_stage_dir: str = "~/clips"

```

```

85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101    # 0 = no extra floor (the headroom guard still applies).
102    min_free_gb: float = 0.0
103    # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104    # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105    # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106    # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107    # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108    # Tri-state:
109    #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110    #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111    #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112    stream_video: Optional[bool] = None
113    # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114    # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115    # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117    # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118    batch_size: Optional[int] = None
119    # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120    # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121    prefetch_batches: Optional[int] = None
122    # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123    # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124    # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
125    # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126    # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
127    # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native

```

```

(Linux)
128         # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
129         # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130         # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131         decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
132
133         @model_validator(mode="after")
134         def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135             # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136             # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137             # healthcheck hard-fails if auto resolves to cpu with this backend on.
138             if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139                 raise ValueError(
140                     f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                     f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142                 )
143             return self
144
145         @field_validator("batch_size")
146         @classmethod
147         def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148             # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149             # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150             # when truthy, so 0 stays "off/parity-default").
151             if v is not None and v < 1:
152                 raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153             return v
154
155         @field_validator("prefetch_batches")
156         @classmethod
157         def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158             # 0 = off (the default). Reject negatives: the runner passes any truthy value as
159             # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160             # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161             if v is not None and v < 0:
162                 raise ValueError(
163                     f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
164                 )
165             return v
166
167
168         class FusionConfig(BaseModel):
169             """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
170
171             Every default reproduces control behaviour: the fusion segmenter persists the
172             shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
173             true ? the person-cue features, but it does NOT gate the served handoff unless a
174             threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
175             is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
176             supplied ? off-court persons (spectators / adjacent courts) are excluded.
177             """
178

```

```

179 # Which trajectory substrate seeds the windows (shuttle stays primary).
180 substrate: Literal["wasb", "tracknet"] = "wasb"
181 # Windowing velocity threshold for the fusion family (the engine reads
182 # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
183 # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
184 velocity_threshold: float = 0.02
185 # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
186 # enabled ? so the default path never imports torch / touches the GPU.
187 cue_flags: dict[str, bool] = Field(default_factory=dict)
188 # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
189 # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
190 min_crossings: int = Field(default=0, ge=0)
191 # Served Gate B: when the person cue is on, drop windows with median in-court
players
192 # < this. 0 = OFF.
193 min_players: int = Field(default=0, ge=0)
194 # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
195 court_polygon: Optional[list[list[float]]] = None
196 # Net line for the person two-sided-motion split (person features only ? the shuttle
197 # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
198 net_axis: Literal["x", "y"] = "y"
199 net_position: float = Field(default=0.5, ge=0.0, le=1.0)
200
201
202 class IndexingConfig(BaseModel):
203     default_segmenter: str = "yolo_hybrid"
204     use_gpu: bool = True
205     # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206     # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207     # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208     # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
210     detector_impl: str = "auto"
211     motion_threshold: float = 0.08
212     min_segment_duration: float = 3.0
213     dead_time_merge_gap: float = 2.0
214     chunk_duration: float = 90.0

```

27. user (2026-07-07T15:13:11.264Z)

```

652 def _execute_processing(
653     config=None, db=None, req=None, progress=None
654 ) -> Dict[str, Any]:
655     """The full hash ? validate ? segment ? report pipeline for one video.
656
657     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
658     module-level ``global_config``/``db_instance`` (backward-compatible with existing
659     tests that pass only ``req``). Route handlers always pass explicit params; older
660     test code that calls ``_execute_processing(req)`` still works.
661
662     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
663     Raises on unexpected internal errors ? the caller records those as a job failure
664     """
665     import backend.main as _main # call-time seam: main's globals/model stay patchable
666
667     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
668     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
669     if config is not None and not isinstance(config, AppConfig):
670         # Old signature: _execute_processing(req, progress=None)
671         if req is None and db is not None and isinstance(db, _main.ProcessRequest):

```

```

672         req = db
673         db = None
674         elif req is None and isinstance(config, _main.ProcessRequest):
675             req = config
676             config = None
677     # Resolve config/db from module globals when not passed explicitly
678     if config is None:
679         config = _main.global_config
680     if db is None:
681         db = _main.db_instance
682
683     notify = progress or (lambda stage: None)
684
685     notify("hashing")
686     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
687     # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
688     # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
689     # reading # consumers (hash / validators / segmenter) use the resolved local path.
690     local_video = ""
691     try:
692         local_video = storage.acquire_local_input(
693             req.video_path, getattr(config, "storage", None)
694         )
695         video_id = _main.compute_video_id(local_video)
696     except Exception:
697         storage.cleanup_acquired_local_input(
698             req.video_path, local_video, getattr(config, "storage", None)
699         )
700         raise
701     try:
702         existing_video = db.get_video(video_id)
703     except Exception:
704         storage.cleanup_acquired_local_input(
705             req.video_path, local_video, getattr(config, "storage", None)
706         )
707         raise
708     was_reingest = existing_video is not None
709
710     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
711     default_seg = getattr(
712         getattr(config, "indexing", None), "default_segmenter", "motion"
713     )
714     seg_name = req.segmenter or default_seg
715     segmenter_actual = None
716     segmentation_ran = False
717     val_results: List[Any] = []
718     val_context: Dict[str, Any] = {}
719     # Compute-path PROVENANCE (the validation method): which backend actually ran the
720     # DETECT ? # "in_process" once the local segmenter is reached, set on the response as
721     # "cloud_run" by # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
722     # fail).
723     compute_actual: Optional[str] = None
724     response: Optional[Dict[str, Any]] = None
725     try:
726         if (
727             existing_video
728             and existing_video.get("status") == "processed"
729             and not req.force
730         ):
731             cached_segments = db.query_play_segments(video_id, {})

```

```

731         if cached_segments:
732             logger.info(
733                 "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
734                 video_id,
735             )
736             response = _cached_process_payload(
737                 video_id,
738                 existing_video.get("validation_results", []),
739                 cached_segments,
740                 getattr(req, "compute", None),
741             )
742             return response
743
744     # 1. Run pluggable validators (with-context variant surfaces fprobe_meta for
the report)
745     notify("validating")
746     logger.info(f"Running validations for {req.video_path}...")
747     val_results, val_context = registry.run_all_with_context(local_video, config)
748     failures = [r for r in val_results if not r.passed]
749
750     if failures:
751         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
752         # status; it no longer destroys the video's prior good segments.
753         db.add_video(
754             video_id,
755             req.video_path,
756             "failed",
757             [r.model_dump() for r in val_results],
758         )
759         response = {
760             "video_id": video_id,
761             "status": "failed",
762             "message": _validation_failure_message(failures),
763             "results": [r.model_dump() for r in val_results],
764         }
765         return response
766
767     # 2. Run segmenter & indexer
768     logger.info("[API] Video passed checks. Segmenting and indexing...")
769     db.add_video(
770         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
771     )
772
773     segmenter_cls = segmenter_registry.get(seg_name)
774     if not segmenter_cls:
775         # Submit-time validation already 400s unknown names; this is the defensive
776         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
777         available = list(segmenter_registry.list_available().keys())
778         response = {
779             "video_id": video_id,
780             "status": "failed",
781             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
782             "results": [r.model_dump() for r in val_results],
783             "play_segments": [],
784         }
785         return response
786
787     logger.info(f"[API] Using segmenter: {seg_name}")
788     notify(f"segmenting ({seg_name})")
789     segmenter_actual = seg_name
790     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only

```

```

791         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
792         play_wm, cand_wm = db.segment_watermarks(video_id)
793         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
794         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
795         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
796         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
797         # they're dropped on the cloud path in this slice ? a documented follow-up.)
798         remote = _maybe_dispatch_remote(
799             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
800         )
801         if remote is not None:
802             response, segmentation_ran = (
803                 remote # `ran` = did the cloud job actually execute (report)
804             )
805             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
806         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
807         compute_actual = "in_process"
808         segmenter = segmenter_cls(db, config)
809         try:
810             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
811             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
812             with _LOCAL_COMPUTE_LANE:
813                 result = segmenter.process_video(
814                     video_id, local_video, req.player_pool, req.max_frames
815                 )
816             except Exception:
817                 # A raising segmenter must not leave half-written rows: restore the pre-run
818                 # state (same destroy-after-confirm contract as the Failure branch), then
let
819                 # the job worker record the failure.
820                 db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
821                 raise

```

28. assistant (2026-07-07T15:14:59.852Z)

Now let me verify the SPA side: whether `wasb\_weights\_present` is consumed anywhere, and confirm the capabilities type shape and picker logic.

29. user (2026-07-07T15:15:03.501Z)

```

=== git status (freshness) ===
fix/hosted-compute-picker-defaults-cloud
1470e2a fix(ingest): default the compute picker to CLOUD on a hosted control-plane
=== wasb_weights_present usages in web/src ===
web/src/api/types.ts:101: wasb_weights_present?: boolean;
=== compute_target usages in web/src ===
web/src/api/types.ts:107: /** `cloud_available` = can THIS server satisfy a per-request
compute="cloud" dispatch (a Cloud Run
web/src/api/types.ts:109: deployment?: { profile?: string; compute_target?: string;
cloud_available?: boolean };
web/src/components/IngestPanel.test.tsx:163: vi.stubGlobal("fetch", fetchAll({ caps: {
deployment: { cloud_available: false } } }));
web/src/components/IngestPanel.test.tsx:174: caps: { deployment: { cloud_available: true
} },
web/src/components/IngestPanel.test.tsx:195: caps: { deployment: { cloud_available: true,
compute_target: "cloud_run" } },

```

```

web/src/components/IngestPanel.test.tsx:376:      caps: { deployment: { cloud_available: true
}, ...capsBucket },
web/src/components/IngestPanel.test.tsx:495:      caps: { deployment: { cloud_available: true
} },
web/src/components/IngestPanel.test.tsx:522:    vi.stubGlobal("fetch", fetchAll({ caps: {
deployment: { cloud_available: true } } }));
web/src/components/IngestPanel.test.tsx:534:    vi.stubGlobal("fetch", fetchAll({ caps: {} }));
// no deployment.cloud_available ? local-only
web/src/components/IngestPanel.test.tsx:83:    // (capabilities.deployment.cloud_available), let the
user choose where DETECT runs. Default LOCAL
web/src/components/IngestPanel.test.tsx:84:    // (free, no surprise cost) EXCEPT on a hosted cloud-
serving control-plane (compute_target=cloud_run):
web/src/components/IngestPanel.test.tsx:105:      setCloudAvailable(c.deployment?.cloud_available
=== true);
web/src/components/IngestPanel.test.tsx:106:      // Hosted cloud-serving control-plane
(compute_target=cloud_run) bakes NO detector weights, so
web/src/components/IngestPanel.test.tsx:109:      // that merely also offers cloud
(compute_target=cloud_run) keeps the free LOCAL default.
web/src/components/IngestPanel.test.tsx:110:      if (c.deployment?.cloud_available === true &&
c.deployment?.compute_target === "cloud_run") {
web/src/components/SetupWizard.test.tsx:41:    deployment: { profile: "cloud-serving",
cloud_available: true },
web/src/components/SetupWizard.test.tsx:120:    stubCapabilities(LIGHT); // no
deployment.cloud_available
web/src/components/SetupWizard.test.tsx:57:    // trusts (capabilities.deployment.cloud_available); on
a local appliance it's false ? true copy.
web/src/components/SetupWizard.test.tsx:58:    const hosted = caps?.deployment?.cloud_available ===
true;

```

30. user (2026-07-07T15:15:04.945Z)

```

=== deployment/capabilities types ===
92:export interface EngineCapabilities {
101:  wasb_weights_present?: boolean;
107:  /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch (a
Cloud Run
109:  deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
113:  storage?: StorageCapabilities;
128:export interface StorageCapabilities {

```

31. assistant (2026-07-07T15:16:40.773Z)

`wasb\_weights\_present` is declared in types.ts but never read by any component ? the SPA picker keys only off `compute\_target`/`cloud\_available`. Let me confirm the stub runner path (a `detector\_impl=stub` box needs no weights) since the guardrail ignores `detector\_impl`.

32. user (2026-07-07T15:16:42.373Z)

```

1      ""CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3      A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4      CSV with no GPU, no WSL, no model weights ? so the whole trajectory-hybrid pipeline
5      (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6      container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7      (``use_gpu=False``); this closes the last GPU/WSL-only gap so the entire pipeline is
8      runnable ? and regression-gateable ? without a GPU.
9
10     Two modes:
11     * replay ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12     pipeline with a committed/golden trajectory; deterministic + cross-OS).
13     * synth (default) ? generate a deterministic synthetic trajectory of
14     ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15     gaps, so windowing yields a stable, known number of candidate windows.
16

```



```

17 This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18 "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19 (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20 ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21 WSL runners stay the default, so production behaviour is unchanged.
22 """
23
24 import csv as _csv
25 import logging
26 import os
27 import shutil
28 from dataclasses import dataclass
29 from typing import Any, List, Optional, Tuple
30
31 from backend.pipeline.detectors.base import DetectorRunner
32
33 # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
34 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
35
36 logger = logging.getLogger(__name__)
37
38
39 @dataclass
40 class StubConfig:
41     """Settings for the CPU stub runner (built from ``config.json`` ->
42     ``indexing.stub``)."""
43
44     fps: float = 30.0
45     frame_width: float = 1280.0
46     num_rallies: int = 2
47     rally_seconds: float = 4.0
48     gap_seconds: float = (
49         3.0 # dead gap between rallies (> merge_gap so windows don't merge)
50     )
51     lead_seconds: float = 2.0 # dead lead-in
52     step_px: float = (
53         12.0 # per-frame shuttle displacement during a rally (>> velocity_thresh)
54     )
55     replay_csv: Optional[str] = (
56         None # if set + exists, emit this CSV verbatim instead of synthesising
57     )
58
59     @classmethod
60     def from_indexing_cfg(cls, idx_cfg: Any) -> "StubConfig":
61         # Stub config has no corresponding section in IndexingConfig ? it's
62         # test-only. Accept IndexingConfig (typed path) or dict (test compat).
63         s: dict = {}
64         if isinstance(idx_cfg, dict):
65             s = idx_cfg.get("stub", {}) or {}
66         # IndexingConfig has no 'stub' field ? use defaults.
67         return cls(
68             fps=float(s.get("fps", 30.0)),
69             frame_width=float(s.get("frame_width", 1280.0)),
70             num_rallies=int(s.get("num_rallies", 2)),
71             rally_seconds=float(s.get("rally_seconds", 4.0)),
72             gap_seconds=float(s.get("gap_seconds", 3.0)),
73             lead_seconds=float(s.get("lead_seconds", 2.0)),
74             step_px=float(s.get("step_px", 12.0)),
75             replay_csv=s.get("replay_csv"),
76         )
77
78 class StubDetectorRunner(DetectorRunner):
79     """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
80

```

```

81     def __init__(self, cfg: Optional[StubConfig] = None):
82         self.cfg = cfg or StubConfig()
83
84     def healthcheck(self) -> Tuple[bool, str]:
85         return True, "stub CPU detector (no GPU/WSL)"
86
87     def _rows(self) -> List[Tuple[int, int, float, float]]:
88         """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight
bursts
89         separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
?
90         well above the default 0.02 velocity threshold ? so every rally frame is
"active"
91         and each burst becomes exactly one candidate window."""
92         cfg = self.cfg
93         rows: List[Tuple[int, int, float, float]] = []
94         frame = 0
95         y = cfg.frame_width / 4.0
96         base_x = cfg.frame_width / 2.0
97         for _ in range(int(cfg.lead_seconds * cfg.fps)):
98             rows.append((frame, 0, 0.0, 0.0))
99             frame += 1
100         for r in range(cfg.num_rallies):
101             for i in range(int(cfg.rally_seconds * cfg.fps)):
102                 x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
103                 rows.append((frame, 1, x, y))
104                 frame += 1
105             if r != cfg.num_rallies - 1:
106                 for _ in range(int(cfg.gap_seconds * cfg.fps)):
107                     rows.append((frame, 0, 0.0, 0.0))
108                     frame += 1
109         return rows
110
111     def run_predict(
112         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
113     ) -> Optional[str]:
114         """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
115         os.makedirs(output_win_dir, exist_ok=True)
116         stem = os.path.splitext(
117             os.path.basename(str(video_win_path).replace("\\", "/"))
118         )[0]
119         key = f"{stem}_{video_id[:12]}" if video_id else (stem or "stub")
120         out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
121
122         if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
123             shutil.copyfile(self.cfg.replay_csv, out_csv)
124             logger.info(
125                 "Stub runner: replayed trajectory %s -> %s",
126                 self.cfg.replay_csv,
127                 out_csv,
128             )
129             return out_csv
130
131         rows = self._rows()
132         with open(out_csv, "w", newline="") as f:
133             w = _csv.writer(f)
134             w.writerow(["Frame", "Visibility", "X", "Y"])
135             for frame, vis, x, y in rows:
136                 w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
137         logger.info(
138             "Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows), out_csv
139         )
140         return out_csv
141

```

```

142     # Model-agnostic CSV parsing + windowing ? identical to the real runners.
143     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
144     trajectory_to_action_windows = staticmethod(
145         TrackNetRunner.trajectory_to_action_windows
146     )
147
148
149     def synth_trajectory_from_labels(
150         labels_csv: str,
151         out_csv: str,
152         *,
153         fps: float = 30.0,
154         frame_width: float = 1280.0,
155         tail_pad: float = 2.0,
156         step_px: float = 12.0,
157     ) -> str:
158         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
159         in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
160         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
161
162         This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
163         no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
164         learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
165         """
166         rallies: List[Tuple[float, float]] = []
167         with open(labels_csv, newline="", encoding="utf-8") as f:
168             for row in _csv.DictReader(f):
169                 try:
170                     rallies.append((float(row["start_time"]), float(row["end_time"])))
171                 except (KeyError, ValueError):
172                     continue
173         duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
174         nframes = max(1, int(duration * fps))
175         base_x = frame_width / 2.0
176         y = frame_width / 4.0
177         os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
178         with open(out_csv, "w", newline="", encoding="utf-8") as f:
179             w = _csv.writer(f)
180             w.writerow(["Frame", "Visibility", "X", "Y"])
181             for i in range(nframes):
182                 t = i / fps
183                 if any(s <= t < e for s, e in rallies):
184                     x = base_x + (step_px if i % 2 else -step_px)
185                     w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
186                 else:
187                     w.writerow([i, 0, "0.000", "0.000"])
188         return out_csv
189

```

33. user (2026-07-07T15:16:42.940Z)

```

90     * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini
key value).
91     * Every field optional/defensive: a missing field degrades the wizard gracefully. */
92     export interface EngineCapabilities {
93         api_version?: number;
94         engine_version?: string;
95         segmenters?: string[];
96         default_segmenter?: string;
97         ffmpeg?: { ffmpeg?: boolean; ffmpeg_path?: string; ffmpeg_path?:

```

```

string };
98     gpu?: { nvidia_smi_present?: boolean };
99     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100     gemini_key_present?: boolean;
101     wasb_weights_present?: boolean;
102     /** The largest upload THIS box accepts, in MB (engine `security.max_upload_mb`,
default 4096).
103     * /upload/init already 413s an oversized file BEFORE any part transfers; the SPA
consumes this to
104     * reject it INSTANTLY client-side (no wasted round-trip). Absent on older engines ?
the guard is
105     * skipped and the engine's server-side fail-fast still applies. */
106     max_upload_mb?: number;
107     /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
108     * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
109     deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
110     /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
111     * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
112     * ? the SPA degrades to local-only (no picker). */
113     storage?: StorageCapabilities;
114     [key: string]: unknown;
115 }
116
117 /** One storable medium reported by GET /api/capabilities (P3). Presence/usable bools
only ? no
118     * secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
119     export interface StorageBackendInfo {

```

34. assistant (2026-07-07T15:18:37.717Z)

I have enough ground truth to complete the fidelity analysis. Let me confirm the key facts I'm relying on before reporting.

Summary of verification:

- Native WASB runner (`native\_wasb\_runner.py`): `weights\_path = env("WASB\_WEIGHTS")` or `indexing.wasb.weights\_path` or `""`, healthcheck does `expanduser` + `exists`. The guard's WASB branch mirrors this **exactly** ? so for the actual production scenario (hosted Linux CP, `detector\_impl=native`, `wasb\_hybrid` / fusion-wasb) the guard is a faithful mirror and correctly fires when weights are absent. No defect there.
- The `/api/capabilities` `wasb\_weights\_present` (main.py:397) does NOT `expanduser`, but the guard DOES (correctly matching the runner), and `wasb\_weights\_present` is declared in `types.ts:101` yet **never read by any SPA component** (the picker keys only off `compute\_target`/`cloud\_available`). So that inconsistency is a latent pre-existing cosmetic gap in the capabilities probe ? not a defect of the guard or the SPA signal. Not reporting it.
- `substrate` is a pydantic `Literal["wasb","tracknet"]` and `cfg` at the call site is the typed `AppConfig` (`global\_config`), so the `.strip().lower()` / dict-vs-typed concerns don't materialize. Not reporting them.
- Confirmed there is **no native TrackNet runner**; `tracknet\_hybrid` uses the base `\_build\_native\_runner` (raises `NotImplementedError`) under `detector\_impl=native`, or the WSL runner under `auto` (needs `wsl`, absent on Linux). The stub runner's healthcheck returns `True` with **no weights**.

The two genuine fidelity gaps are the tracknet/fusion-tracknet escape-hatch unsoundness (false-ALLOW) and the guard ignoring `detector\_impl` (false-BLOCK).

35. user (2026-07-07T15:18:57.470Z)

Structured output provided successfully